

CIA-3 PROJECT DEVELOPMENT — SUBMISSION REPORT

Christ University • 5th Semester

Field	Detail
Project Code & Title	P19 — Online Grocery Delivery Platform
Domain	Quick Commerce / Grocery
Course Name	Backend Development with Node.js, Express, and Mongo
Section / Batch	5 BTCS B
Semester	5th Semester
Team Number	19

Team Details

S.No	Student Name	Roll No.	Department	Section
1	A Aaryan Dharmmik	2460301	B.Tech CSE	5 BTCS B
2	Abhishek R	2460307	B.Tech CSE	5 BTCS B
3	Adriel B John	2460314	B.Tech CSE	5 BTCS B
4	Abhay Job K J	2460485	B.Tech CSE	5 BTCS B

GitHub Repository: <https://github.com/Aaryan658/L-T-5th-sem-grocery-delivery-platform>

The repository is public and contains the complete Node.js + Express.js + MongoDB source code, a README.md (project title, team details, module list, setup steps, API endpoints), a .gitignore, a .env.example, and the exported Postman collection. node_modules and .env are excluded from version control.

Table of Contents

Table of Contents.....	2
1. Project Overview.....	4
1.1 Problem Statement.....	4
1.2 Objectives.....	4
1.3 Scope & Boundaries.....	4
1.4 Technology Stack	4
1.5 Summary of Implemented Modules	5
2. System Architecture.....	6
2.1 Request Pipeline	6
2.2 Project Structure	6
2.3 Response Envelope	6
3. Database Design.....	7
3.1 Collections.....	7
3.2 Reference vs. Embed Reasoning	8
3.3 Indexes	8
3.4 Entity Relationships.....	8
3.5 Order Status State Machine.....	9
4. Functional Modules & Evidence	10
4.1 Module 1 — User Registration & Authentication.....	10
4.2 Module 2 — Dark-Store Management	12
4.3 Module 3 — Product Catalog & Store-Wise Stock.....	13
4.4 Module 4 — Order Placement with Nearest-Store Check.....	14
4.5 Module 5 — Order Picking & Packing Workflow	16
4.6 Module 6 — Delivery Partner Assignment	17
4.7 Module 7 — Delivery Status Tracking.....	18
4.8 Module 8 — Real-Time Order Status for Customer.....	20
4.9 Module 9 — Stock Replenishment Alerts	22
4.10 Module 10 — Delivery Time Slot Selection.....	23
4.11 Module 11 — Customer Order History & Reorder	24
4.12 Module 12 — Store & Delivery Performance Reports.....	25
4.13 Module 13 — Role-Based Access Control.....	26
5. Complete API Endpoint Reference.....	29

5.1 Auth.....	29
5.2 Dark-Stores & Store Stock.....	29
5.3 Products	29
5.4 Orders	29
5.5 Delivery Partners, Slots & Admin Reports	30
6. Testing, Validation & Error Handling	31
6.1 Negative & Edge Cases.....	31
6.2 Validation & Error-Handling Design	32
7. Setup & Run Instructions	34
7.1 Seed Accounts.....	34
8. Known Limitations	34
9. Learnings & Challenges	35
10. Conclusion.....	35

1. Project Overview

The Online Grocery Delivery Platform (P19) is a backend-focused quick-commerce system. It lets customers order daily essentials for home delivery, lets dark-store staff pick and pack those orders from live per-store inventory, and lets delivery partners handle last-mile delivery — all tracked through one unified order pipeline with role-based access for four roles: Customer, Store Staff, Delivery Partner, and Admin.

The system is delivered as a REST API built with Node.js and Express.js, backed by MongoDB (Mongoose ODM). Authentication uses JSON Web Tokens with bcrypt password hashing; every request body is validated with Joi before it reaches business logic; and every response follows a single JSON envelope. A small vanilla HTML/CSS/JS frontend is included to demonstrate every role's flow live, and a Postman collection covers every endpoint.

1.1 Problem Statement

Quick-commerce grocery delivery needs a backend that lets customers order daily essentials for home delivery, lets dark-store staff pick and pack those orders from live per-store inventory, and lets delivery partners handle last-mile delivery, all tracked through one unified order pipeline with role-based access for Customer, Store Staff, Delivery Partner, and Admin.

1.2 Objectives

- Provide secure registration and login for four distinct user roles using JWT and hashed passwords.
- Manage multiple dark-stores, a shared product catalog, and independent per-store stock levels.
- Resolve a customer's nearest serviceable store by pincode and validate stock before an order is accepted.
- Model the full order lifecycle as an explicit, guarded state machine (placed to picking to packed to assigned to out-for-delivery to delivered).
- Let store staff run a picking/packing queue and let delivery partners update live delivery status.
- Give customers real-time order status with a complete status-history timeline, order history, and one-tap reorder.
- Support delivery time-slot selection, low-stock replenishment alerts, and an admin performance report.
- Enforce access control at the route level (by role) and at the record level (a user only sees orders it is party to).

1.3 Scope & Boundaries

In scope: the complete order pipeline, inventory, RBAC, slots, alerts and reporting as REST endpoints, plus a demo UI and Postman collection. Out of scope (not mocked beyond the nearest-store-by-pincode lookup): online payments, SMS/email notifications, and maps/geolocation routing.

1.4 Technology Stack

Layer	Technology
Runtime / Framework	Node.js 18+, Express.js 4
Database / ODM	MongoDB, Mongoose 7
Authentication	jsonwebtoken (JWT), bcrypt password hashing
Validation	Joi schemas via a validate() middleware on every write route
API testing / docs	Postman collection (all endpoints, auto-captured token)
Demo frontend	Vanilla HTML / CSS / JavaScript, served statically by Express
Config	dotenv (.env), central config/constants.js for business constants

1.5 Summary of Implemented Modules

All 13 planned modules are implemented and working (minimum required: 12).

#	Module	Status
1	User Registration & Authentication	Implemented
2	Dark-Store Management	Implemented
3	Product Catalog & Store-Wise Stock	Implemented
4	Order Placement with Nearest-Store Check	Implemented
5	Order Picking & Packing Workflow	Implemented
6	Delivery Partner Assignment	Implemented
7	Delivery Status Tracking	Implemented
8	Real-Time Order Status for Customer	Implemented
9	Stock Replenishment Alerts	Implemented
10	Delivery Time Slot Selection	Implemented
11	Customer Order History & Reorder	Implemented
12	Store & Delivery Performance Reports	Implemented
13	Role-Based Access Control	Implemented

2. System Architecture

The backend is a layered Express application. A request flows through the layers top to bottom, and any error at any layer is converted to the standard JSON error envelope by a single central error handler.

2.1 Request Pipeline

```
HTTP request
-> CORS + express.json() body parsing
-> Route (routes/*.js)      maps URL + method to a controller
-> protect (middleware/auth.js) verifies JWT, loads live user -> req.user
-> authorize(...roles)      route-level role allow-list (RBAC)
-> validate(schema)         Joi validates + strips the request body
-> Controller (controllers/*.js) business logic, record-level checks
-> Model (models/*.js)      Mongoose schema + indexes -> MongoDB
-> apiResponse.success/error single JSON envelope
-> notFound + errorHandler  catch-all -> clean JSON, never a stack trace
```

2.2 Project Structure

```
backend/
  config/      db.js (Mongo connection), constants.js (business constants)
  models/      Mongoose schemas, one file per collection
  routes/      Express routes, grouped by resource
  controllers/ business logic for each route
  middleware/  auth.js, authorize.js, validate.js, errorHandler.js, notFound.js
  validators/  Joi schemas per resource
  utils/       jwt, hash, apiResponse, asyncHandler, slots, ApiError
  seed.js      demo data (2 stores, 6 products, stock, 1 user per role)
  server.js    app entry point, wires every route
frontend/     optional static demo UI (served at /)
postman/      exported Postman collection covering every endpoint
docs/         ER notes, team split, screenshot runbook
```

2.3 Response Envelope

Every endpoint returns one of these two shapes, so the client never has to special-case a route:

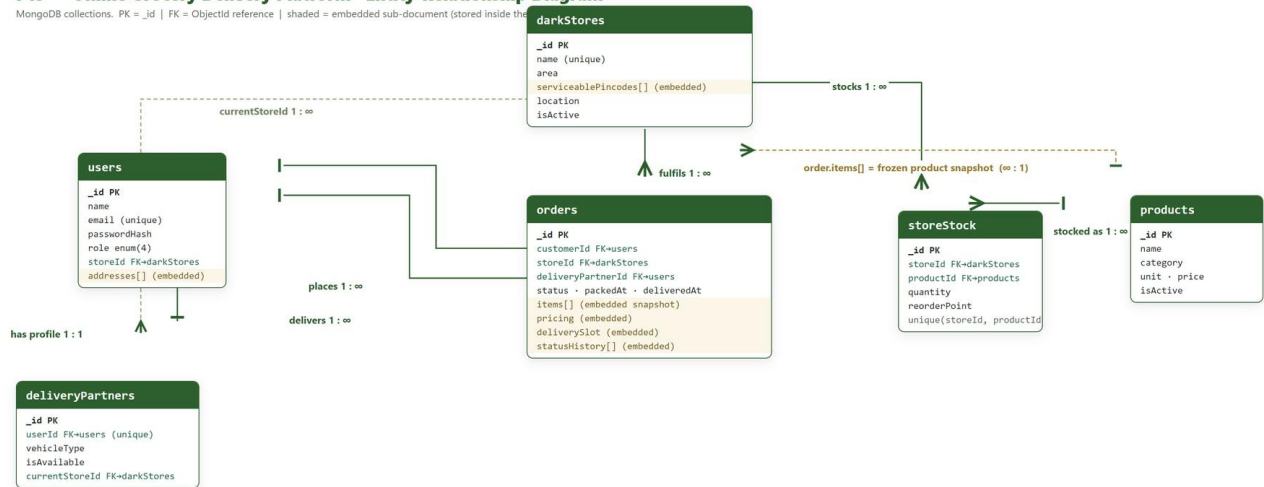
```
{ "success": true, "message": "...", "data": { ... } }
{ "success": false, "message": "...", "errorCode": "VALIDATION_ERROR" }
```

3. Database Design

MongoDB with six collections. The design deliberately mixes referencing and embedding: shared, independently-updated entities are referenced by ObjectId; small sub-documents that are always read with their parent (and, for orders, must be frozen as a point-in-time snapshot) are embedded.

P19 — Online Grocery Delivery Platform • Entity-Relationship Diagram

MongoDB collections. PK = `_id` | FK = ObjectId reference | shaded = embedded sub-document (stored inside the parent)



Referenced (own collection, joined by ObjectId): users, darkStores, products, storeStock, deliveryPartners, and orders.customerId / storeId / deliveryPartnerId. **Embedded** (shaded, stored in the parent): order.items[] / pricing / deliverySlot / statusHistory[], user.addresses[], darkStore.serviceablePincodes[], order.items[] keeps a frozen price/name copy of the product (gold dashed) so a later catalog change never rewrites a past invoice.

Figure 3.1 — Entity-Relationship diagram. PK = `_id`; FK = ObjectId reference; shaded rows are embedded sub-documents; cardinality shown as 1 : many.

3.1 Collections

Collection	Purpose	Key Fields
users	Every account: customer, storeStaff, deliveryPartner, admin	name, email (unique), passwordHash, role, storeId (staff only), addresses[] (embedded)
darkStores	Fulfilment locations	name (unique), area, serviceablePincodes[] (embedded), location, isActive
products	Master catalog, store-agnostic	name, category, unit, price, isActive
storeStock	Per-store quantity for each product	storeId (ref), productId (ref), quantity, reorderPoint
orders	The unified order pipeline	customerId/storeId/deliveryPartnerId (ref), items[] (embedded snapshot), pricing (embedded), deliverySlot (embedded), status, statusHistory[] (embedded), packedAt, deliveredAt
deliveryPartners	Operational profile for a deliveryPartner user	userId (ref, unique), vehicleType, isAvailable, currentStoreId (ref)

3.2 Reference vs. Embed Reasoning

Data	Choice	Why
order.items[]	Embedded	Always read with the order; price/name frozen as a snapshot of the moment of purchase so a later catalog price change never rewrites a past invoice; never queried on their own.
order.pricing, order.deliverySlot	Embedded	Small, single-purpose, only ever meaningful attached to their one order.
order.statusHistory[]	Embedded	Small append-only audit trail; always read with the order to render its timeline (Module 8).
user.addresses[]	Embedded	Small, rarely updated independently, always needed inline when placing an order.
darkStore.serviceablePincodes[]	Embedded	The nearest-store check needs the whole list in one read; edited only alongside the store.
storeStock (storeId, productId)	Referenced	Large in aggregate, queried independently of both parents, updated on its own every order; embedding would rewrite a whole store/product doc per quantity change.
deliveryPartners.userId	Referenced	isAvailable is write-hot; assignment queries scan this collection alone without the rest of the user profile.
order.customerId / storeId / deliveryPartnerId	Referenced	Shared across many orders, updated independently of any single order.

3.3 Indexes

Collection	Index	Reason
users	{ email: 1 } unique	Enforces uniqueness, speeds up login lookups
users	{ role: 1, storeId: 1 }	'staff of store X' lookups
darkStores	{ name: 1 } unique	Prevents duplicate store names
darkStores	{ serviceablePincodes: 1 }	Nearest-store-by-pincode check
products	{ name: 1 }, { category: 1 }	Catalog browsing / search
storeStock	{ storeId: 1, productId: 1 } unique	One stock row per store/product pair; blocks duplicates under a race
storeStock	{ productId: 1 }	'which stores carry this product' queries
orders	{ customerId: 1, createdAt: -1 }	Customer order history (Module 11)
orders	{ storeId: 1, status: 1, createdAt: -1 }	Store picking queue + performance report (Modules 5, 12)
orders	{ deliveryPartnerId: 1, status: 1 }	A partner's active deliveries (Modules 6, 7)
deliveryPartners	{ userId: 1 } unique	One partner profile per user
deliveryPartners	{ isAvailable: 1, currentStoreId: 1 }	Assignment queries (Module 6)

3.4 Entity Relationships

```

users (role=customer)      places      >-- orders
users (role=storeStaff)    belongs to --< darkStores
users (role=deliveryPartner) has      --1 deliveryPartners profile
darkStores                 stocks      >-- storeStock rows
darkStores                 fulfils     >-- orders
products                   appears in  >-- storeStock rows
products                   snapshotted >-- order.items[] (embedded copy, not a live ref)
orders                     assigned to --1 deliveryPartners (via deliveryPartnerId ->
users)

orders embeds: items[], pricing, deliverySlot, statusHistory[]

```

3.5 Order Status State Machine

Every transition is declared in config/constants.js (ORDER_STATUS_FLOW). The controller rejects any transition not in the allow-list with 409 INVALID_TRANSITION, so a caller can never push an order into an illegal state.

```

placed          -> picking | cancelled
picking         -> packed | cancelled
packed          -> assigned | cancelled
assigned        -> out-for-delivery | cancelled
out-for-delivery -> delivered | failed
delivered / failed / cancelled -> (terminal)

```

4. Functional Modules & Evidence

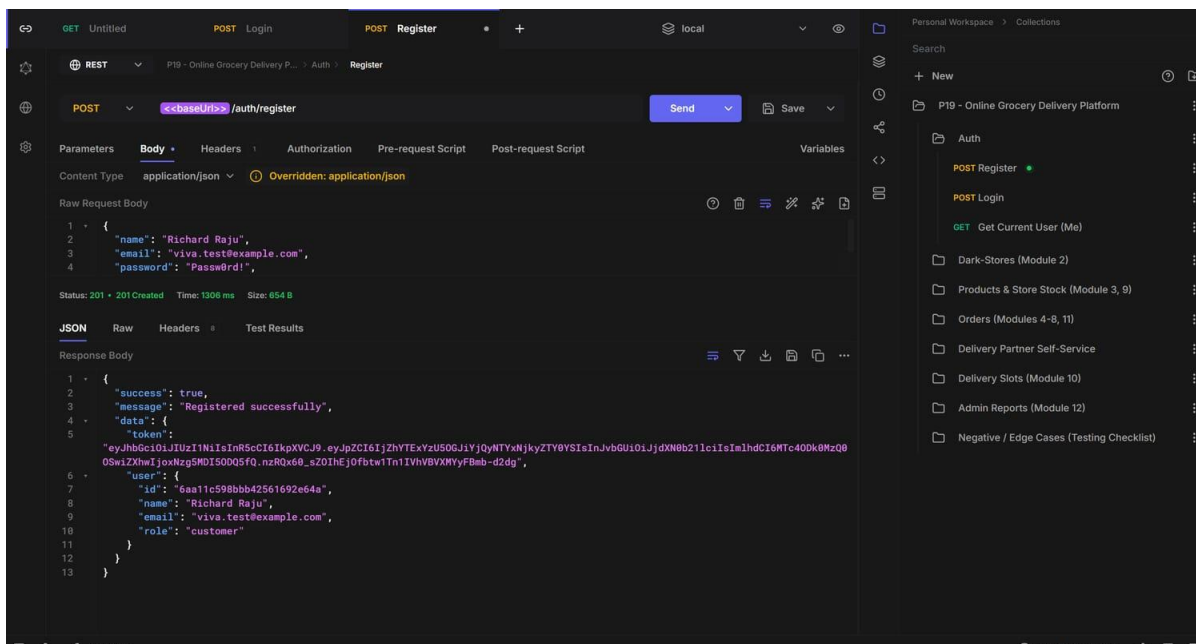
Each module below lists its purpose, its endpoints, and a request/response screenshot captured against the live backend (Postman) and/or the demo UI.

4.1 Module 1 — User Registration & Authentication

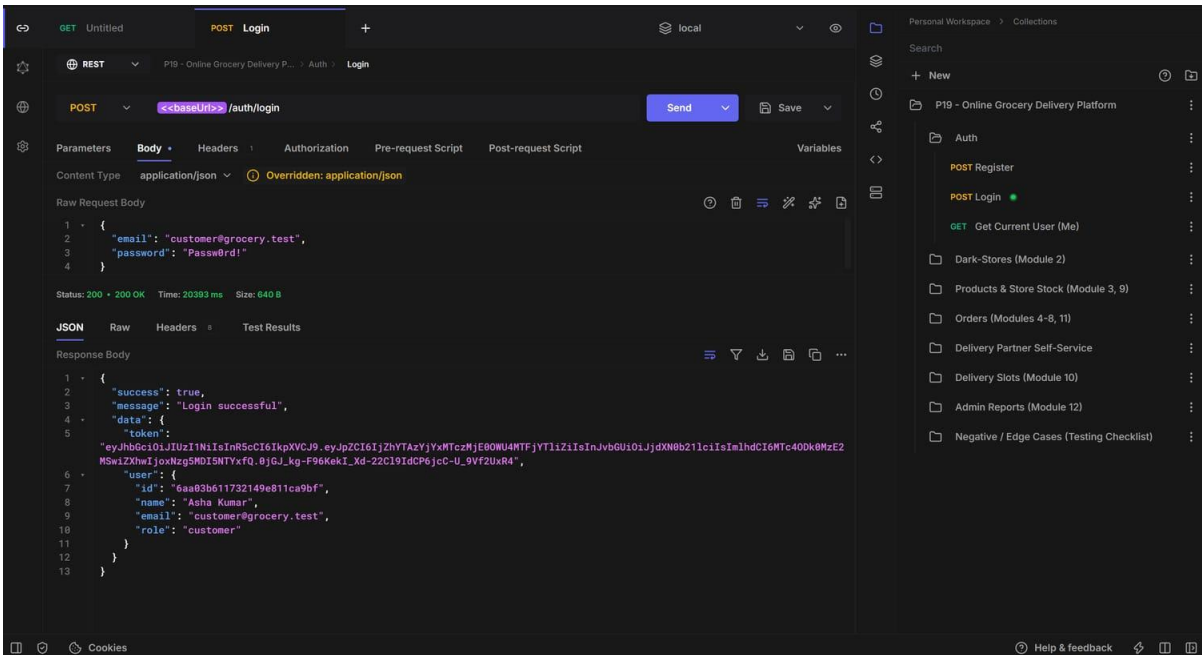
Public register and login endpoints. Passwords are bcrypt-hashed (passwordHash, select:false). Login and register both return a signed JWT ({ id, role }) that every protected route consumes. Registering as a deliveryPartner also auto-creates the matching deliveryPartners operational profile. GET /auth/me returns the live user record decoded from the token.

Endpoints:

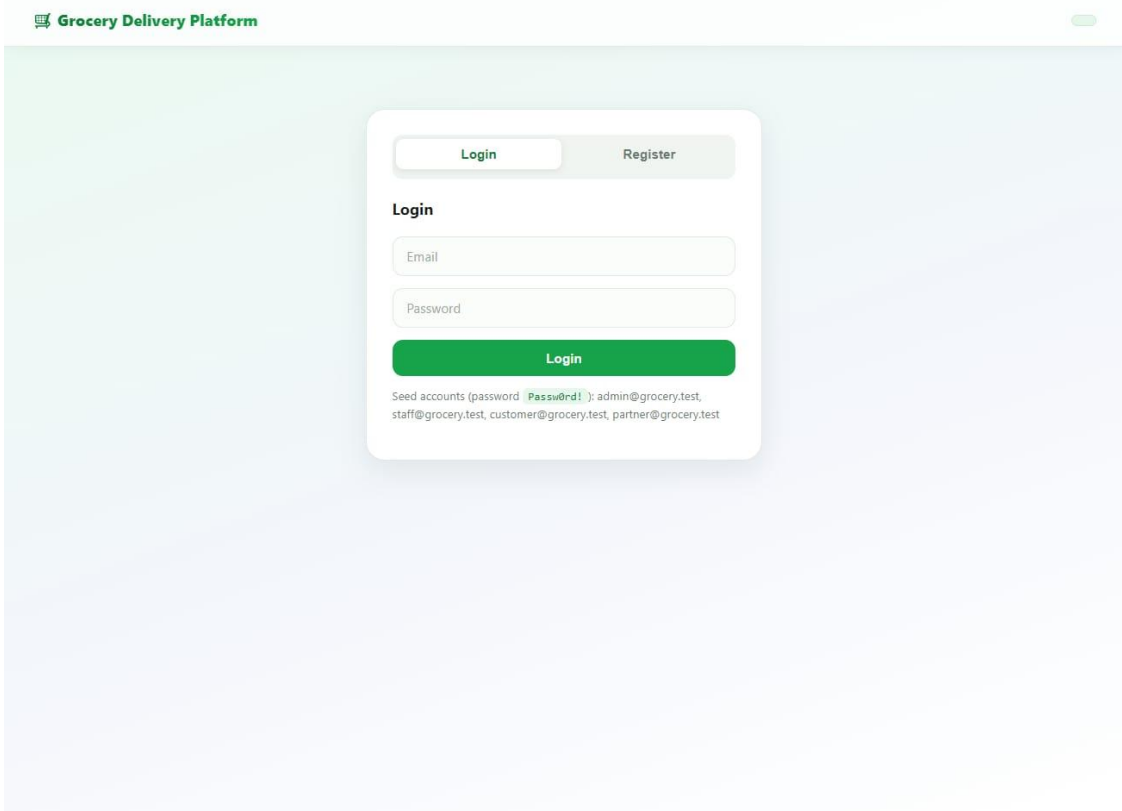
POST /api/auth/register	Public	Register customer / staff / partner / admin
POST /api/auth/login	Public	Login, returns JWT
GET /api/auth/me	Any	Current authenticated user



Postman — POST /api/auth/register -> 201 Registered successfully (token returned)



Postman — POST /api/auth/login -> 200 Login successful; token auto-captured



Demo UI — login screen with seed-account hint

Grocery Delivery Platform

Login Register

Register

Full name

Email

Password (min 6 chars)

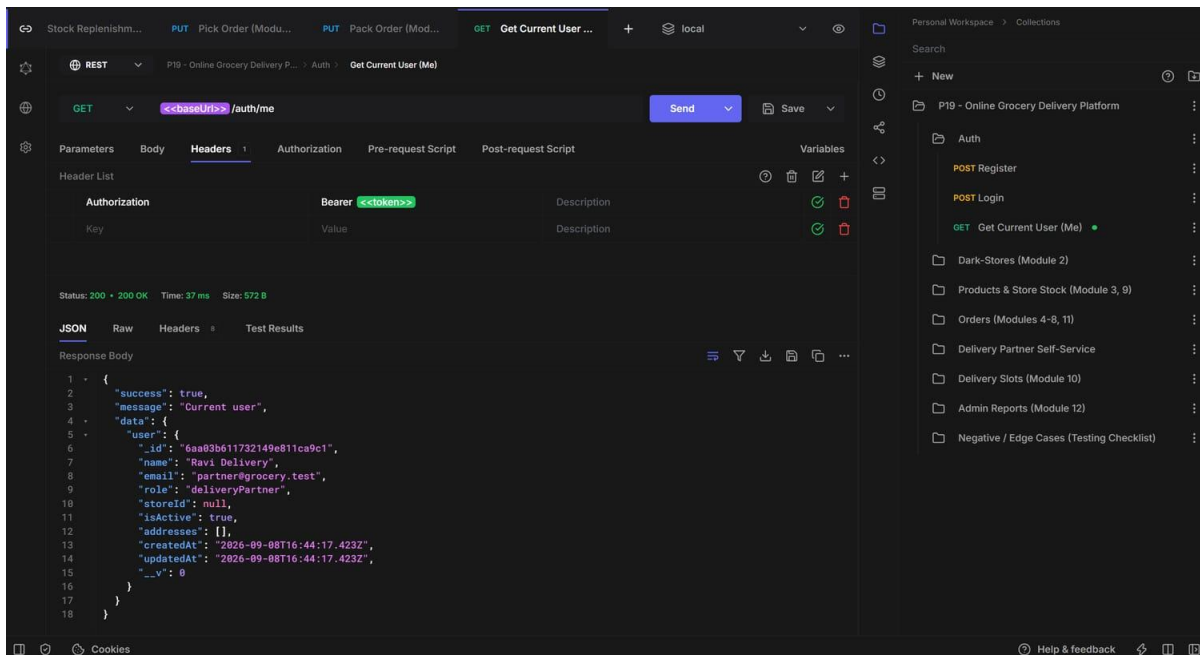
Phone (optional)

Customer

Store ID (storeStaff only)

Register

Demo UI — registration form (role selector for all four roles)



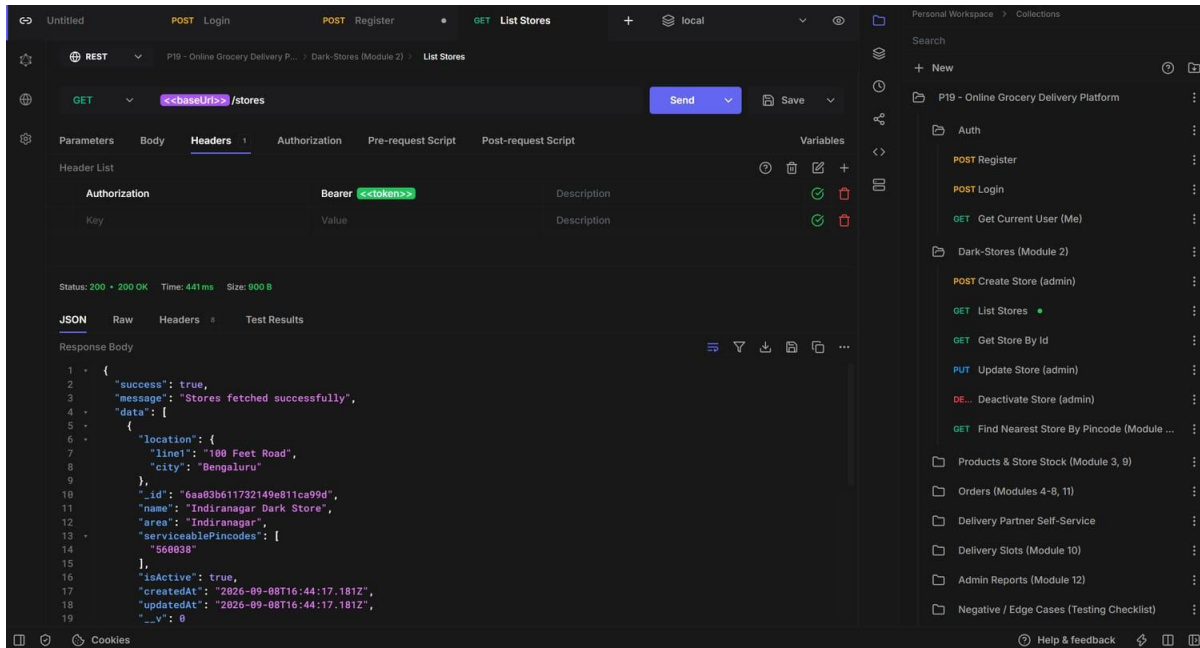
Postman — GET /api/auth/me -> 200 Current user

4.2 Module 2 — Dark-Store Management

Admin CRUD over dark-stores. Browsing is open to any authenticated user so a customer can resolve their nearest store; writes are admin-only. Delete is a soft delete (isActive:false) so historical orders and stock rows stay intact.

Endpoints:

POST	/api/stores	Admin	Create a dark-store
GET	/api/stores	Any	List dark-stores
GET	/api/stores/:id	Any	Get one store
PUT	/api/stores/:id	Admin	Update a store
DELETE	/api/stores/:id	Admin	Deactivate a store (soft delete)
GET	/api/stores/nearest?pincode=	Any	Nearest-store check by pincode



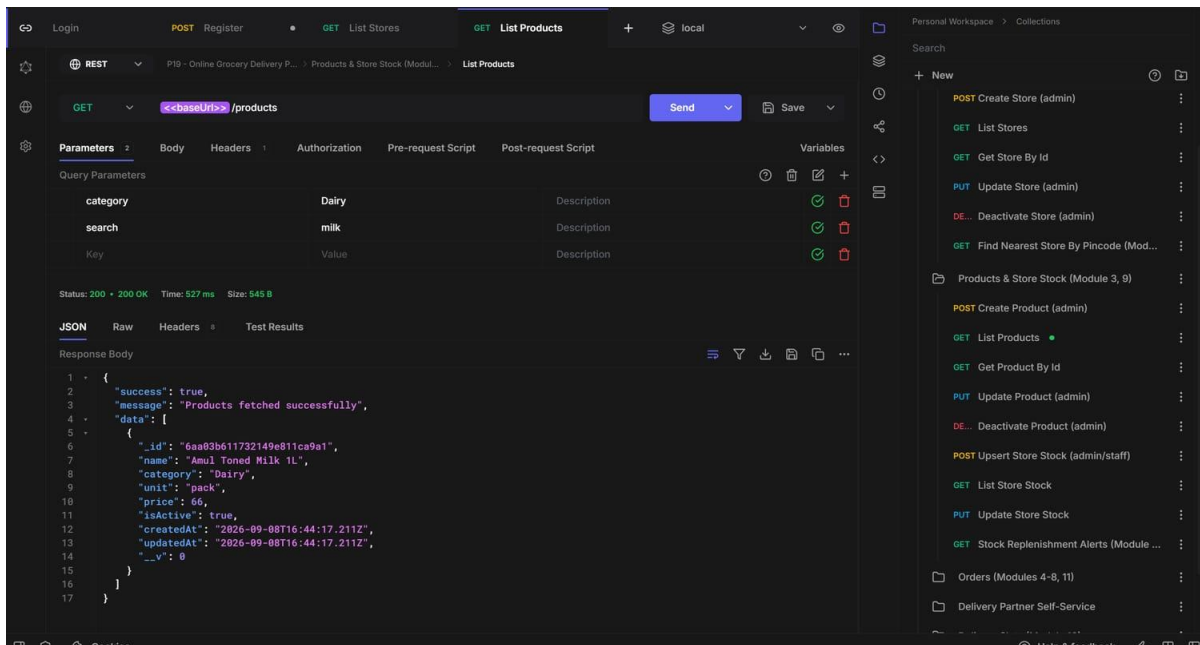
Postman — GET /api/stores -> 200 Stores fetched successfully

4.3 Module 3 — Product Catalog & Store-Wise Stock

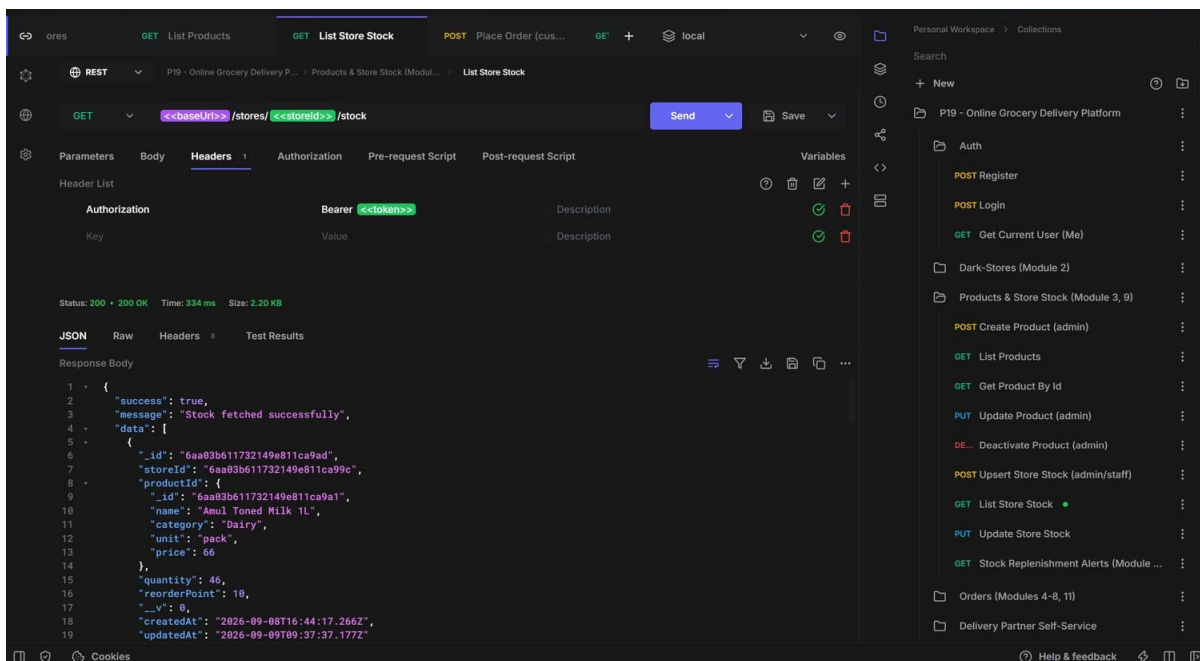
A store-agnostic master catalog (products) plus a separate storeStock collection holding quantity and reorderPoint for each (store, product) pair. Catalog browse/search is open to any authenticated role; catalog writes are admin-only; stock is managed by admin and the staff of that store. A unique compound index on (storeId, productId) guarantees exactly one stock row per pair.

Endpoints:

GET	/api/products?category=&search=	Any	Browse / search products
POST	/api/products	Admin	Create a product
GET	/api/stores/:storeId/stock	Admin/Staff	List a store's stock
POST	/api/stores/:storeId/stock	Admin/Staff	Upsert stock for a product
PUT	/api/stores/:storeId/stock/:productId	Admin/Staff	Update quantity / reorderPoint



Postman — GET /api/products -> 200 Products fetched successfully



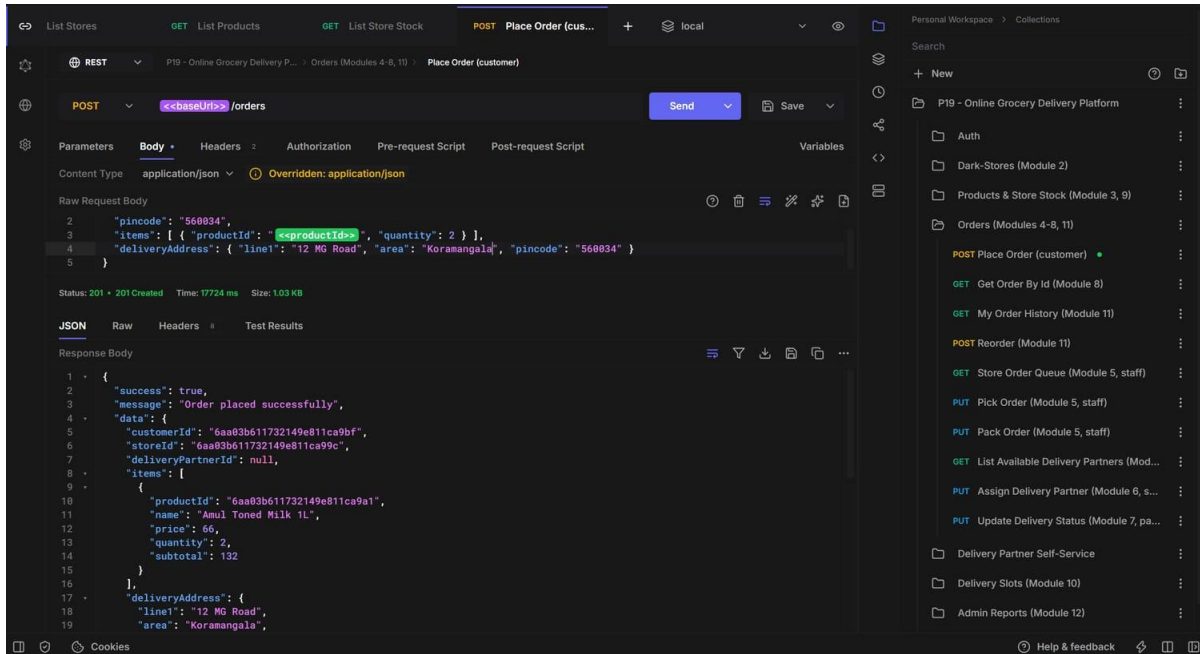
Postman — GET /api/stores/:storeId/stock -> 200 Stock fetched successfully (6 rows)

4.4 Module 4 — Order Placement with Nearest-Store Check

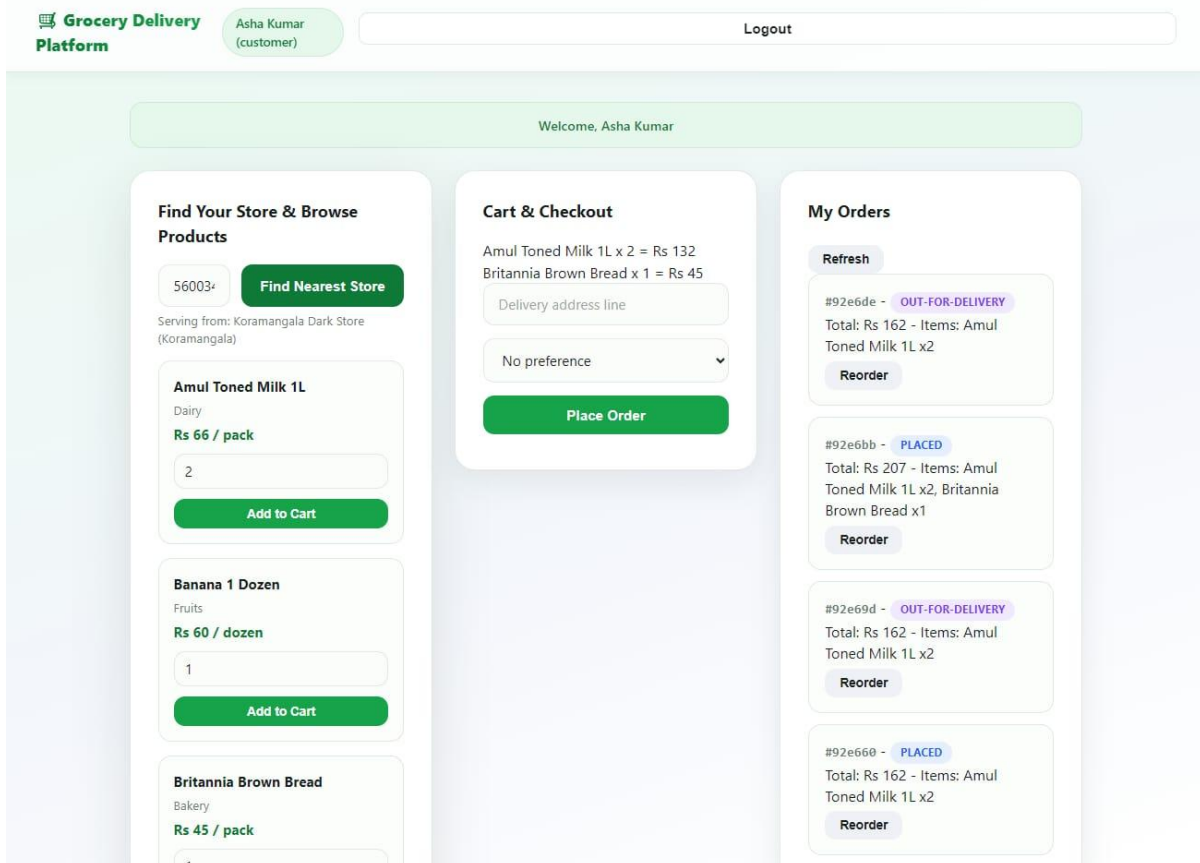
Customer-only. The controller (a) optionally validates the chosen delivery slot, (b) resolves the serviceable store for the pincode, (c) checks live stock for every requested line item at that store and rejects the whole order if any item is short (409 OUT_OF_STOCK, with a message naming the product, store, requested and available quantity), (d) computes pricing (flat Rs 30 delivery fee, free above Rs 499), (e) decrements stock, and (f) writes the order with a frozen item snapshot and an initial 'placed' status event.

Endpoints:

POST /api/orders Customer Place an order (nearest-store + stock check)



Postman — POST /api/orders -> 201 Order placed successfully (itemsTotal 132, deliveryFee 30, total 162)



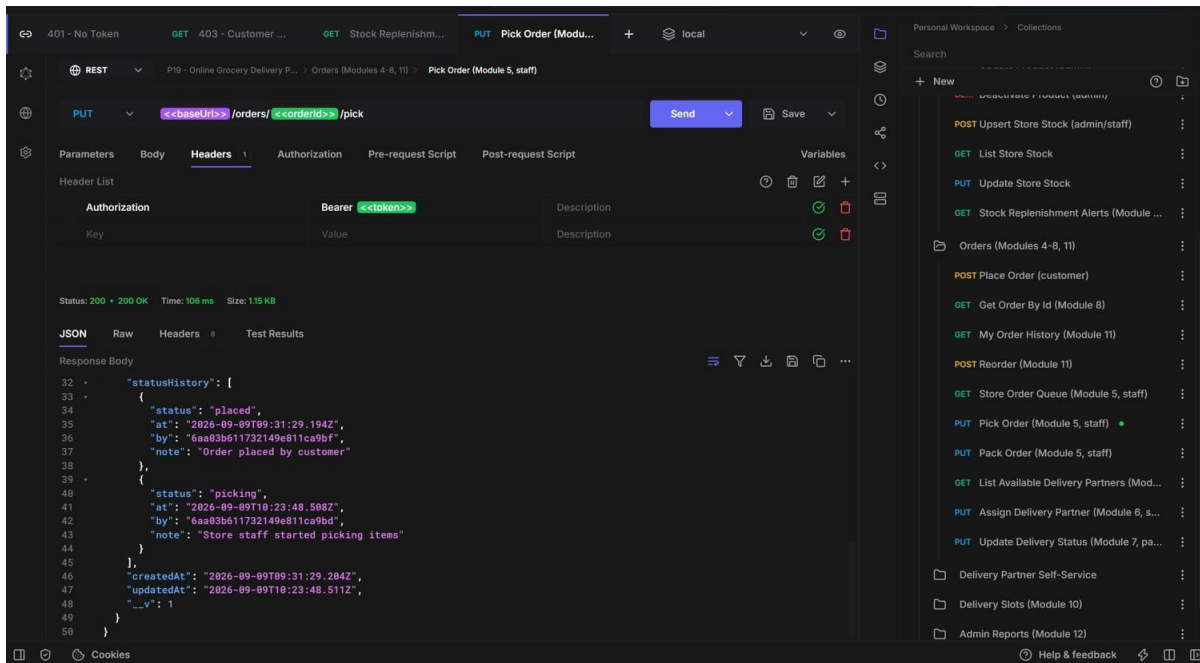
Demo UI — customer finds nearest store by pincode, browses products, builds a cart and checks out

4.5 Module 5 — Order Picking & Packing Workflow

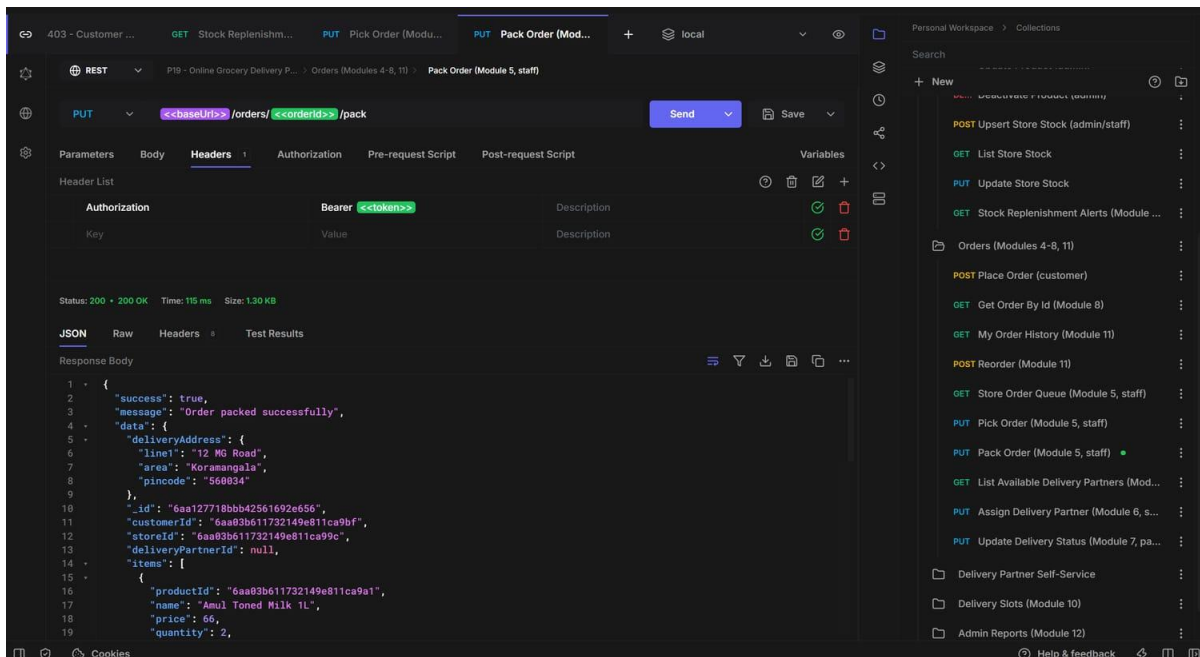
Store staff (and admin) move an order through picking -> packed via two guarded transitions. Packing stamps packedAt (used later by the performance report). Staff also have a per-store order queue filtered by status. Every transition is checked against ORDER_STATUS_FLOW and against the record-level canAccessOrder guard (staff can only act on their own store's orders).

Endpoints:

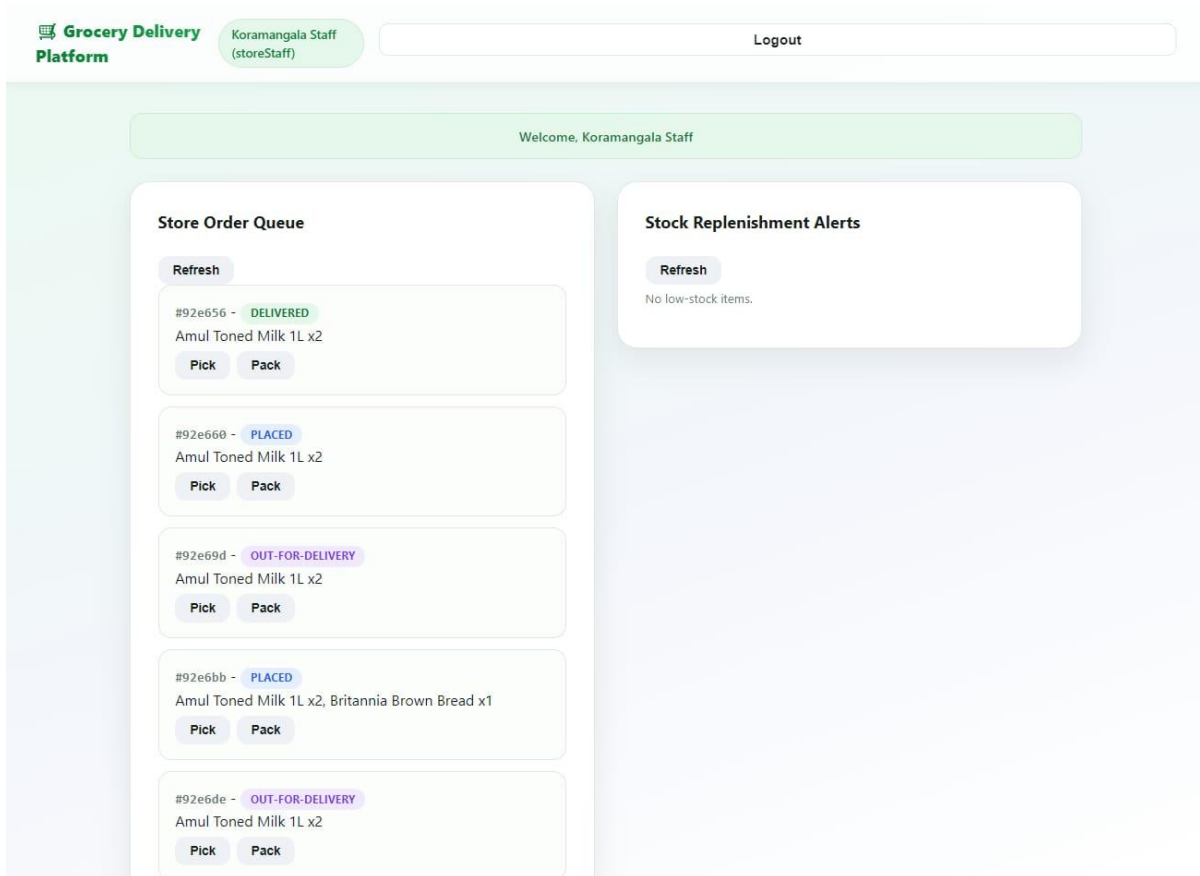
GET /api/stores/:storeId/orders?status=	Admin/Staff	Store picking / packing queue
PUT /api/orders/:id/pick	Staff/Admin	Mark items being picked (-> picking)
PUT /api/orders/:id/pack	Staff/Admin	Mark order packed (-> packed)



Postman — PUT /api/orders/:id/pick -> 200 Order moved to picking



Postman — PUT `/api/orders/:id/pack` -> 200 Order packed successfully (packedAt set)



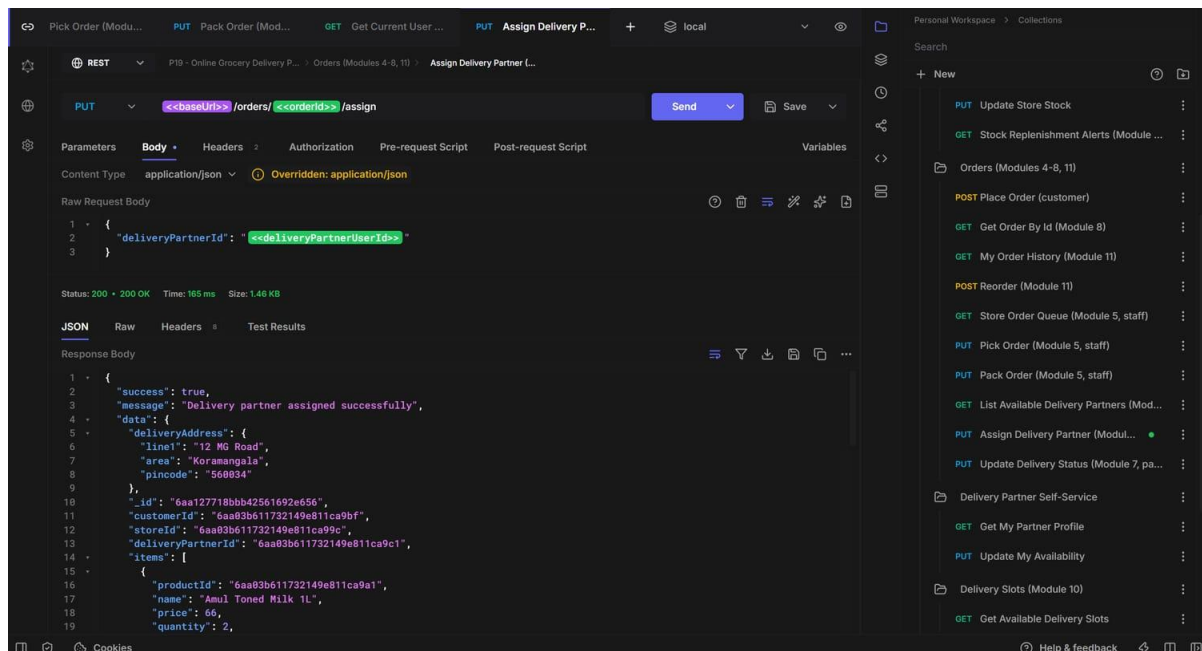
Demo UI — store staff order queue with Pick / Pack actions and status pills

4.6 Module 6 — Delivery Partner Assignment

Staff/admin dispatch a packed order to an available delivery partner. The controller verifies the target transition (packed -> assigned), that the partner profile exists, and that `isAvailable` is true (else 409 PARTNER_UNAVAILABLE); it then sets `order.deliveryPartnerId`, flips the partner to unavailable, and records the partner's current `storeId`. Supporting endpoints list available partners and let a partner toggle their own availability.

Endpoints:

GET /api/delivery-partners/available?storeId=	Staff/Admin	List available partners
GET /api/delivery-partners/me	Partner	My partner profile
PUT /api/delivery-partners/me/availability	Partner	Toggle availability
PUT /api/orders/:id/assign	Staff/Admin	Assign a delivery partner (-> assigned)



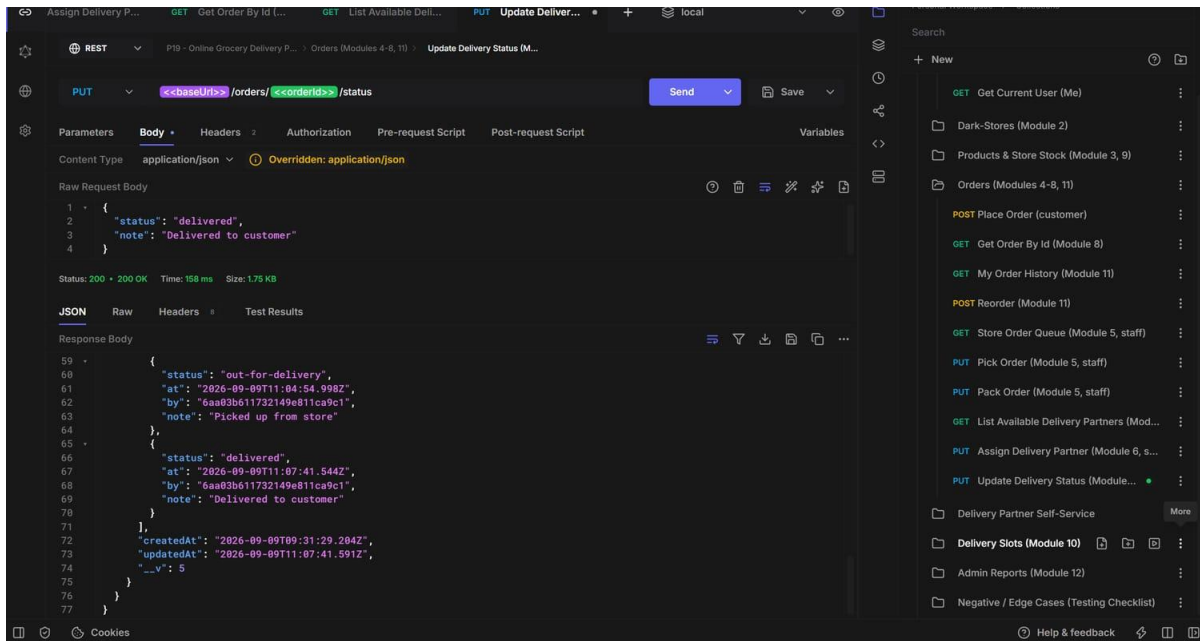
Postman — PUT /api/orders/:id/assign -> 200 Delivery partner assigned successfully (status: assigned)

4.7 Module 7 — Delivery Status Tracking

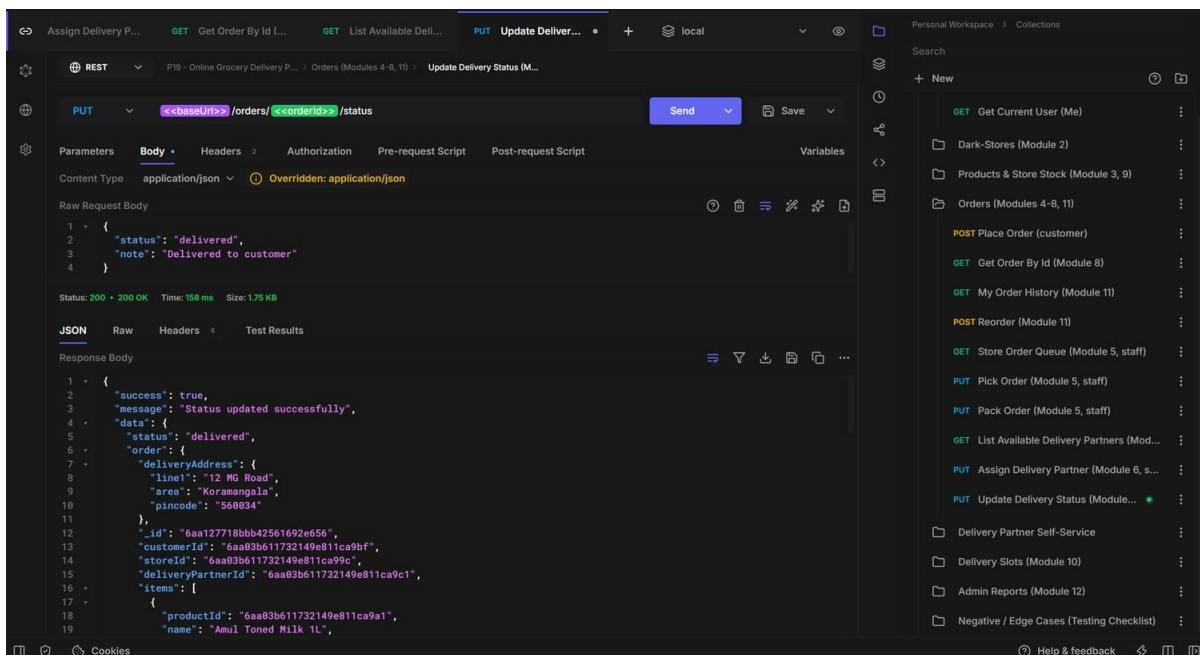
The assigned delivery partner (or admin) advances the order assigned -> out-for-delivery -> delivered (or -> failed). Reaching a terminal state stamps `deliveredAt` and frees the partner (`isAvailable` back to true). Illegal jumps are rejected with 409 INVALID_TRANSITION.

Endpoints:

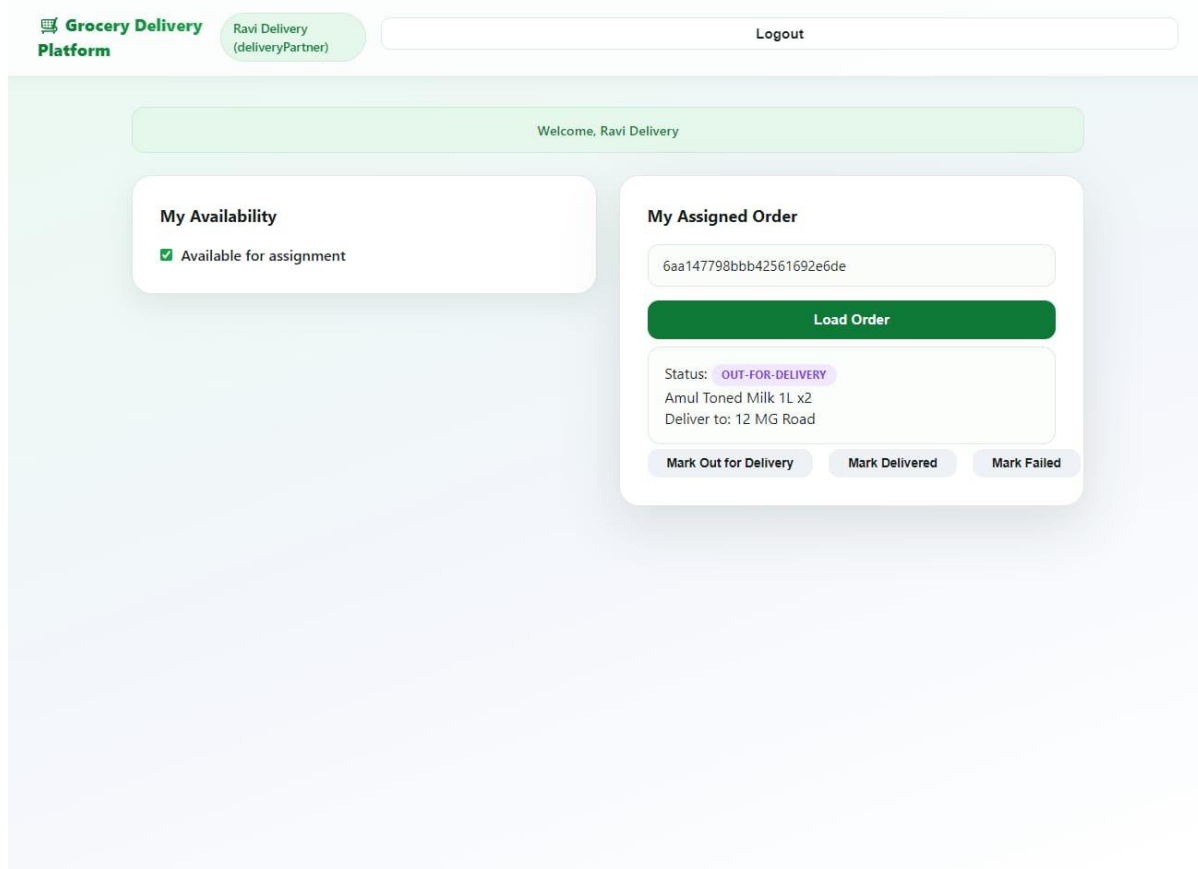
PUT /api/orders/:id/status	Partner/Admin	Update delivery status	{ status, note }
----------------------------	---------------	------------------------	------------------



Postman — PUT `/api/orders/:id/status` -> 200 status: out-for-delivery



Postman — PUT `/api/orders/:id/status` -> 200 status: delivered (deliveredAt set)



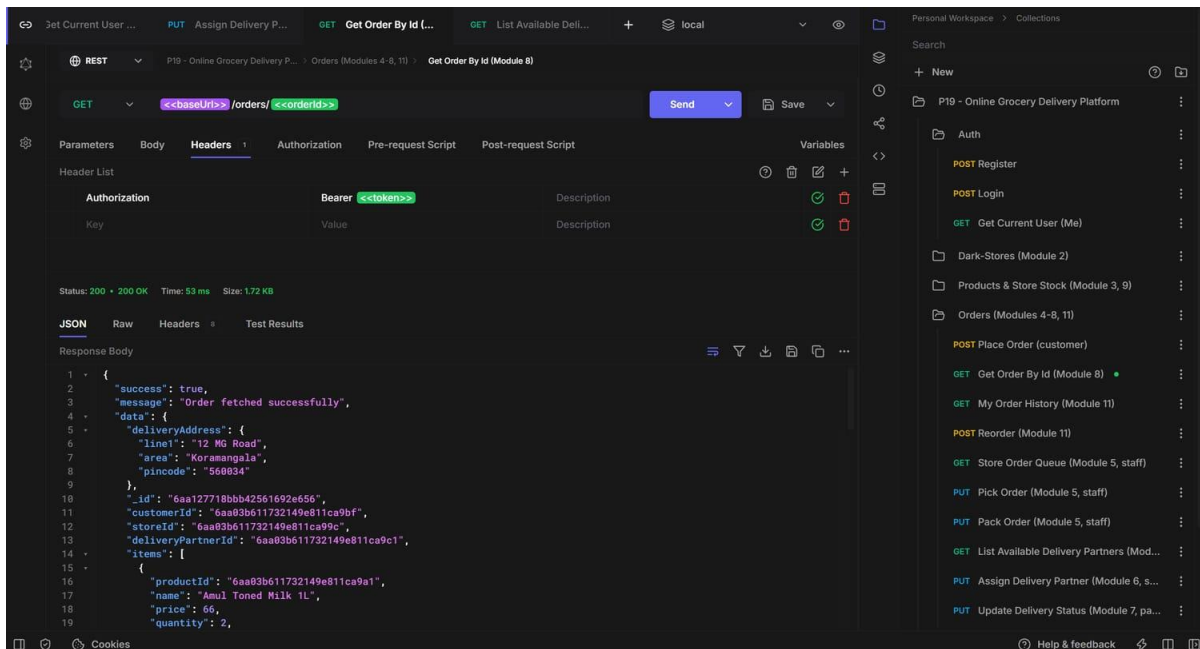
Demo UI — delivery partner loads the assigned order and updates its status

4.8 Module 8 — Real-Time Order Status for Customer

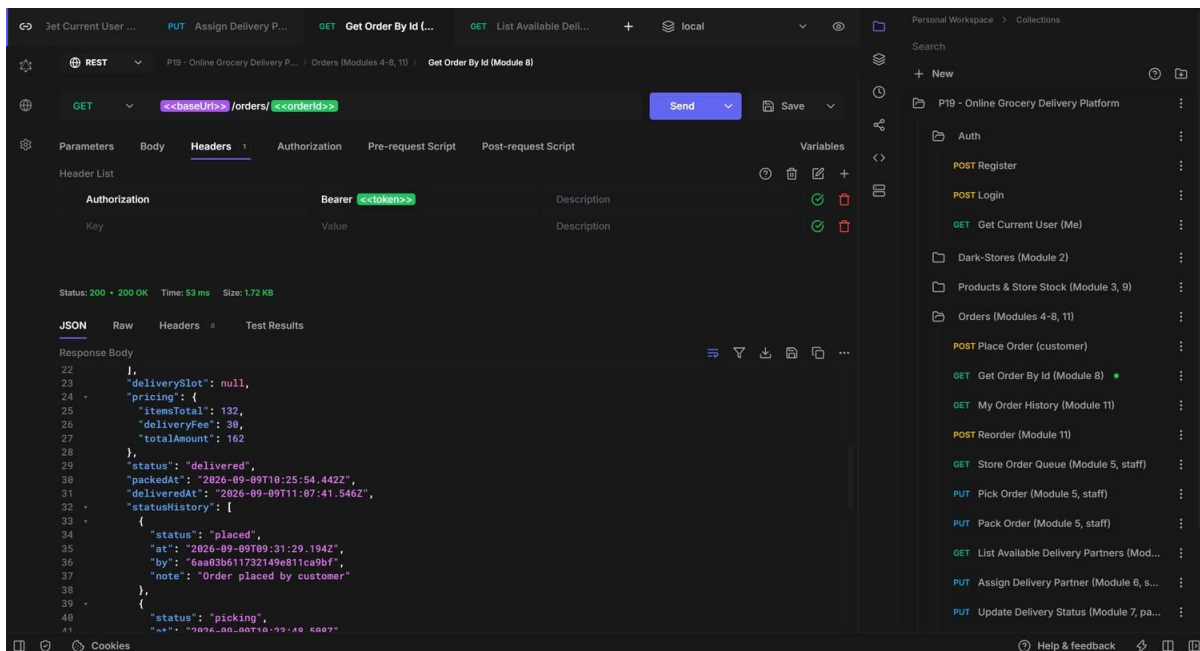
GET `/api/orders/:id` returns the order plus its full `statusHistory[]` — an append-only list of `{ status, at, by, note }` events — so the customer app can render a live timeline. Access is checked inside the controller: a customer sees only their own orders, staff only their store's, a partner only the order assigned to them, admin everything.

Endpoints:

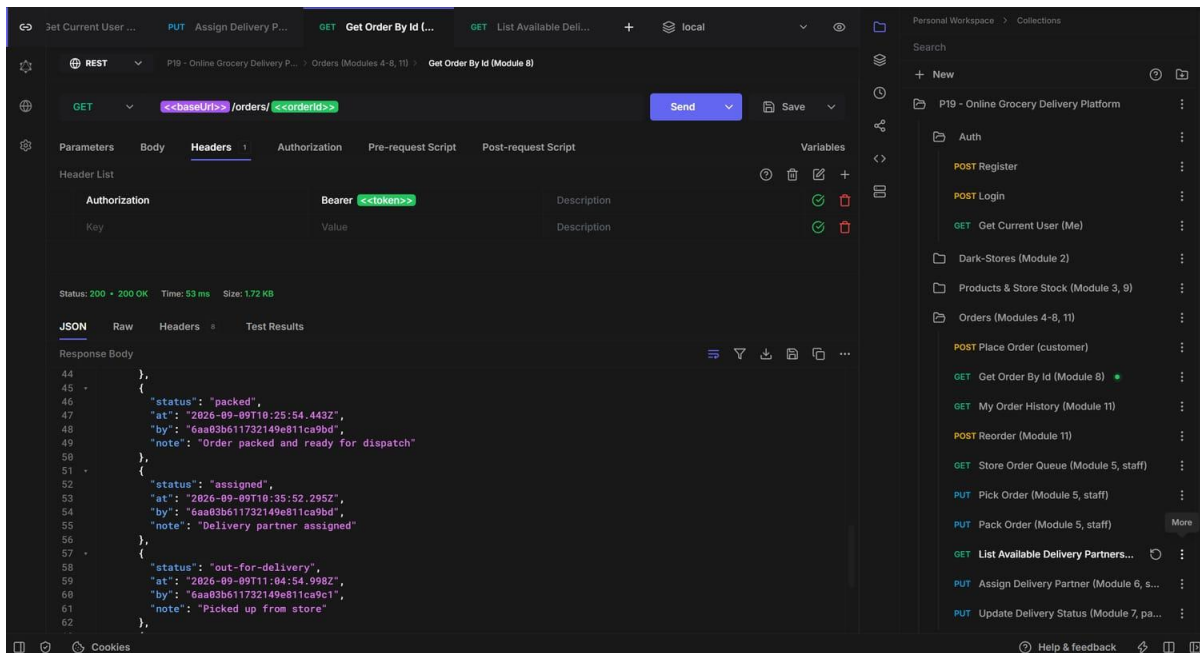
GET <code>/api/orders/:id</code>	Owner / Staff / Partner / Admin	Real-time status + full history
----------------------------------	---------------------------------	---------------------------------



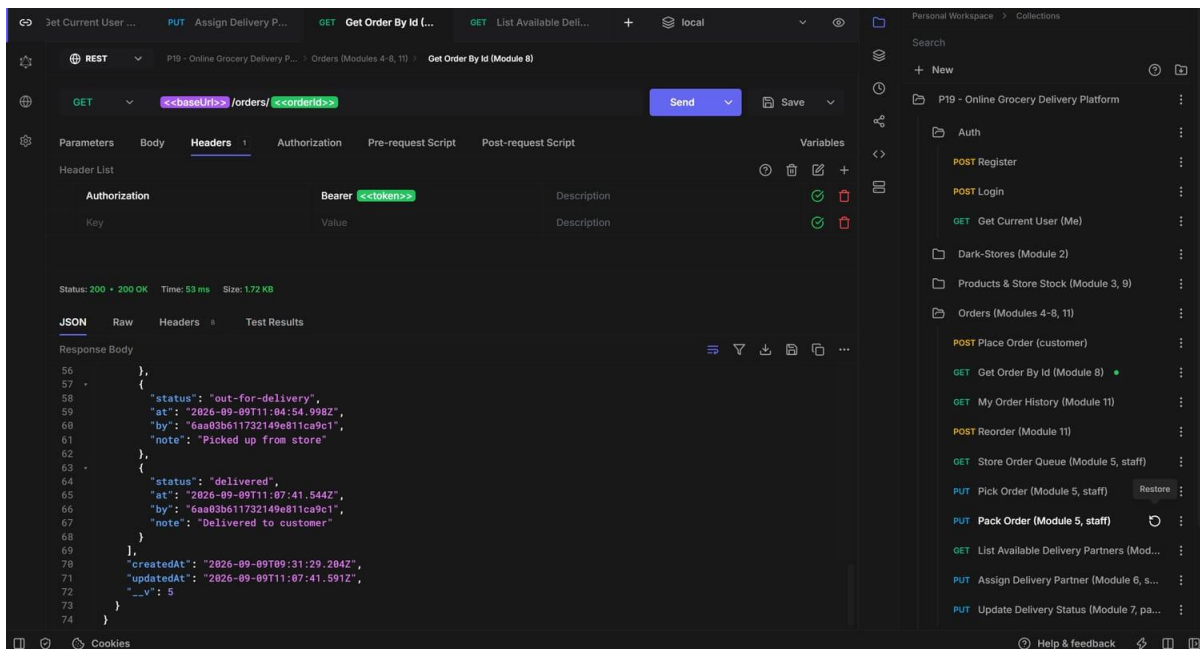
Postman — GET /api/orders/:id -> 200, status history (1/4): placed -> picking



Postman — full timeline (2/4): packed -> assigned



Postman — full timeline (3/4): out-for-delivery



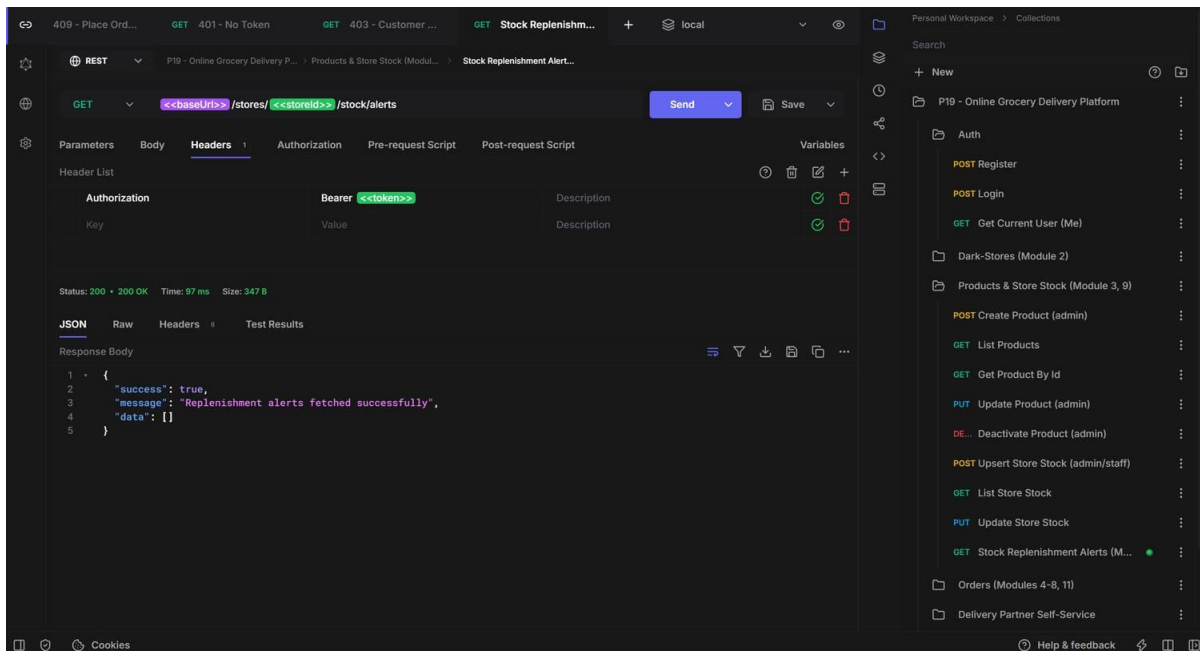
Postman — full timeline (4/4): delivered — all six events present

4.9 Module 9 — Stock Replenishment Alerts

GET `/api/stores/{storeId}/stock/alerts` returns every stock row where quantity $\leq$ reorderPoint (default reorderPoint 10), populated with product name/category/unit, so staff know what to restock.

Endpoints:

GET `/api/stores/{storeId}/stock/alerts` Admin/Staff Items at or below reorderPoint



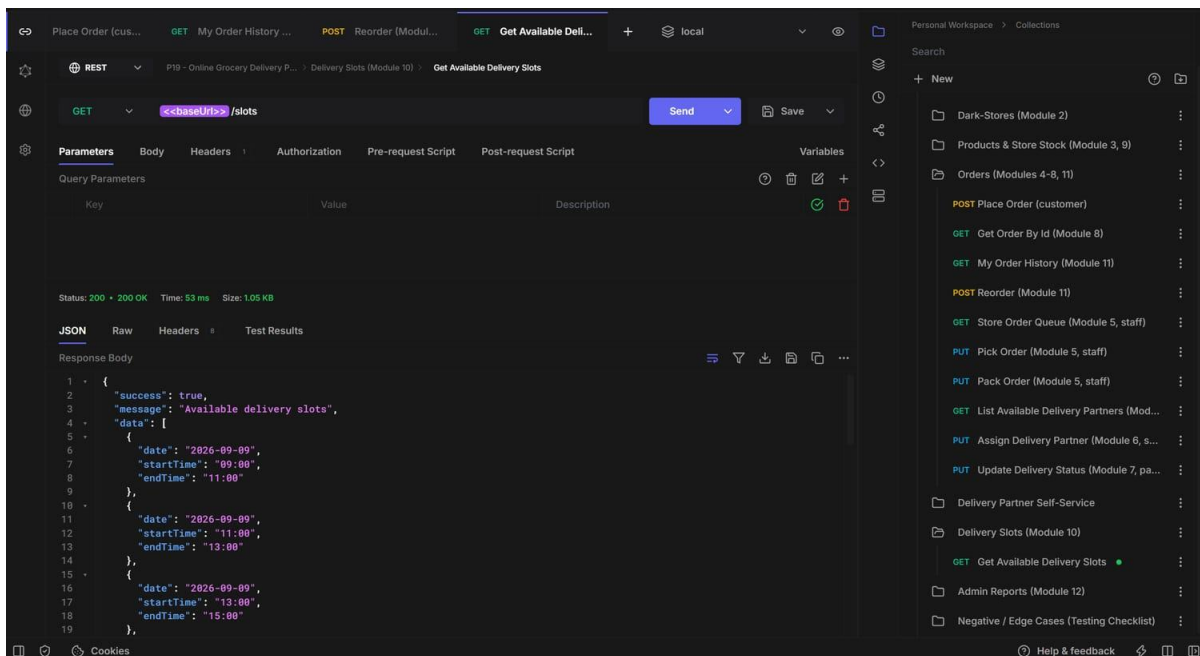
Postman — GET /api/stores/:storeId/stock/alerts -> 200 Replenishment alerts fetched successfully

4.10 Module 10 — Delivery Time Slot Selection

GET /api/slots generates bookable two-hour windows for today and tomorrow from DELIVERY_SLOT_WINDOWS (09:00-21:00, six windows/day = 12 slots). A slot passed at checkout is validated against this same generator before the order is accepted.

Endpoints:

GET /api/slots Any Bookable delivery windows (today + tomorrow)



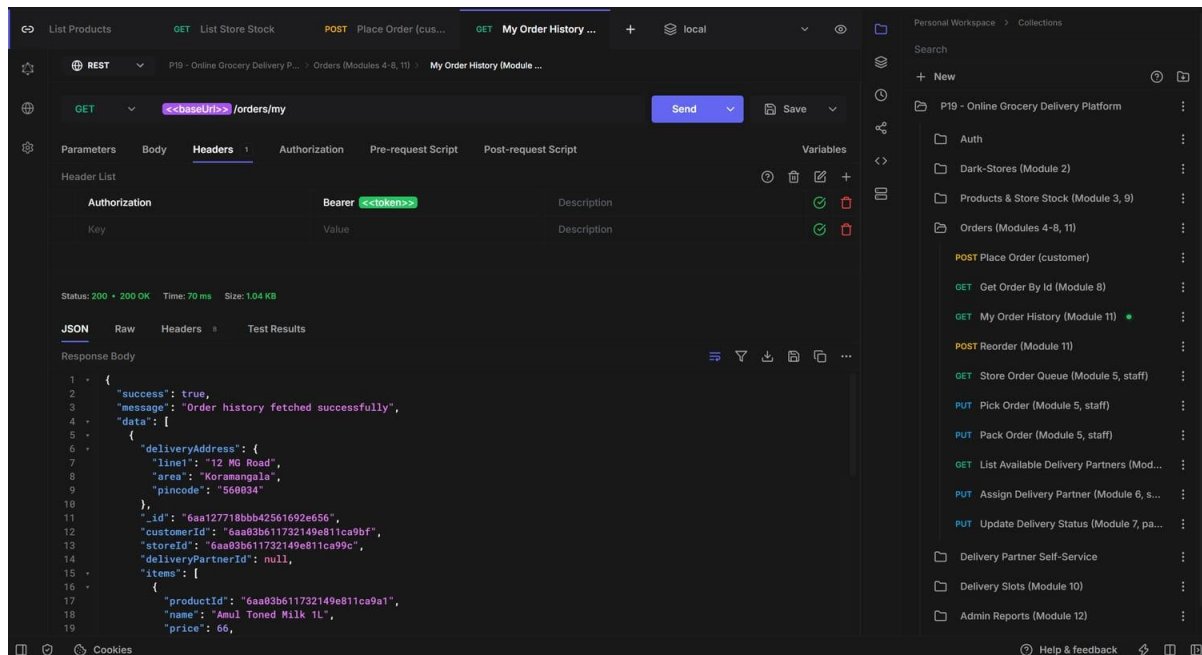
Postman — GET /api/slots -> 200 Available delivery slots (12 windows)

4.11 Module 11 — Customer Order History & Reorder

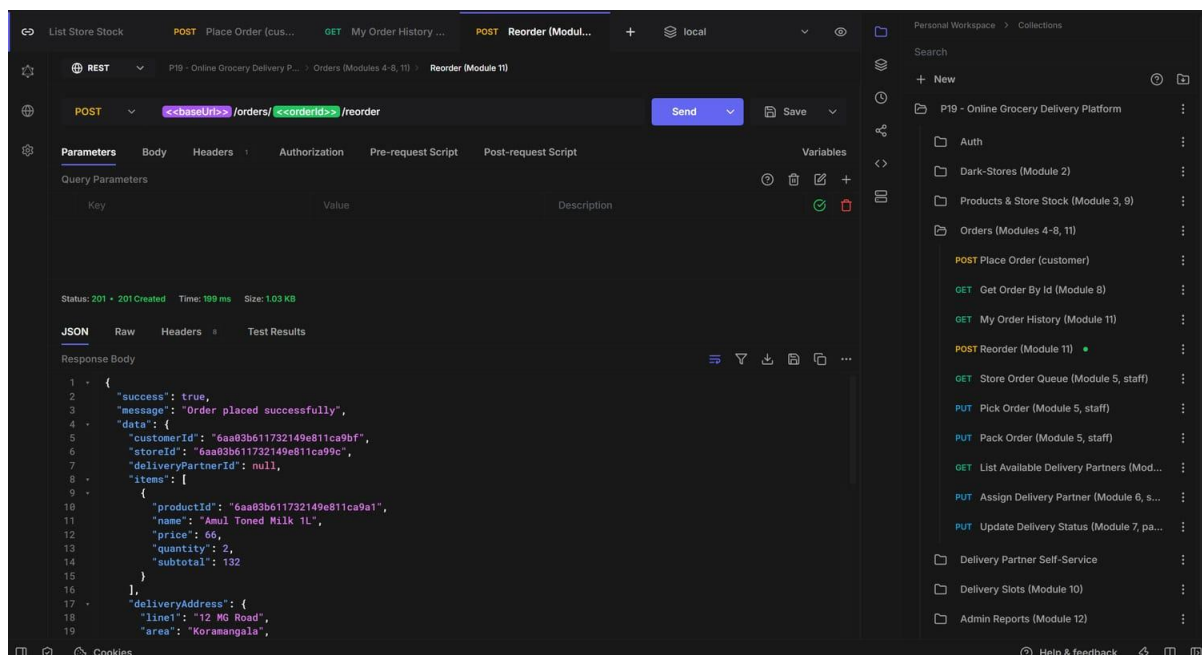
GET /api/orders/my lists the customer's orders newest-first (served by the { customerId, createdAt:-1 } index). POST /api/orders/:id/reorder rebuilds the request from a past order and re-runs the full placement path — so current stock and current prices are re-validated rather than blindly replaying an old snapshot; it returns a brand-new order.

Endpoints:

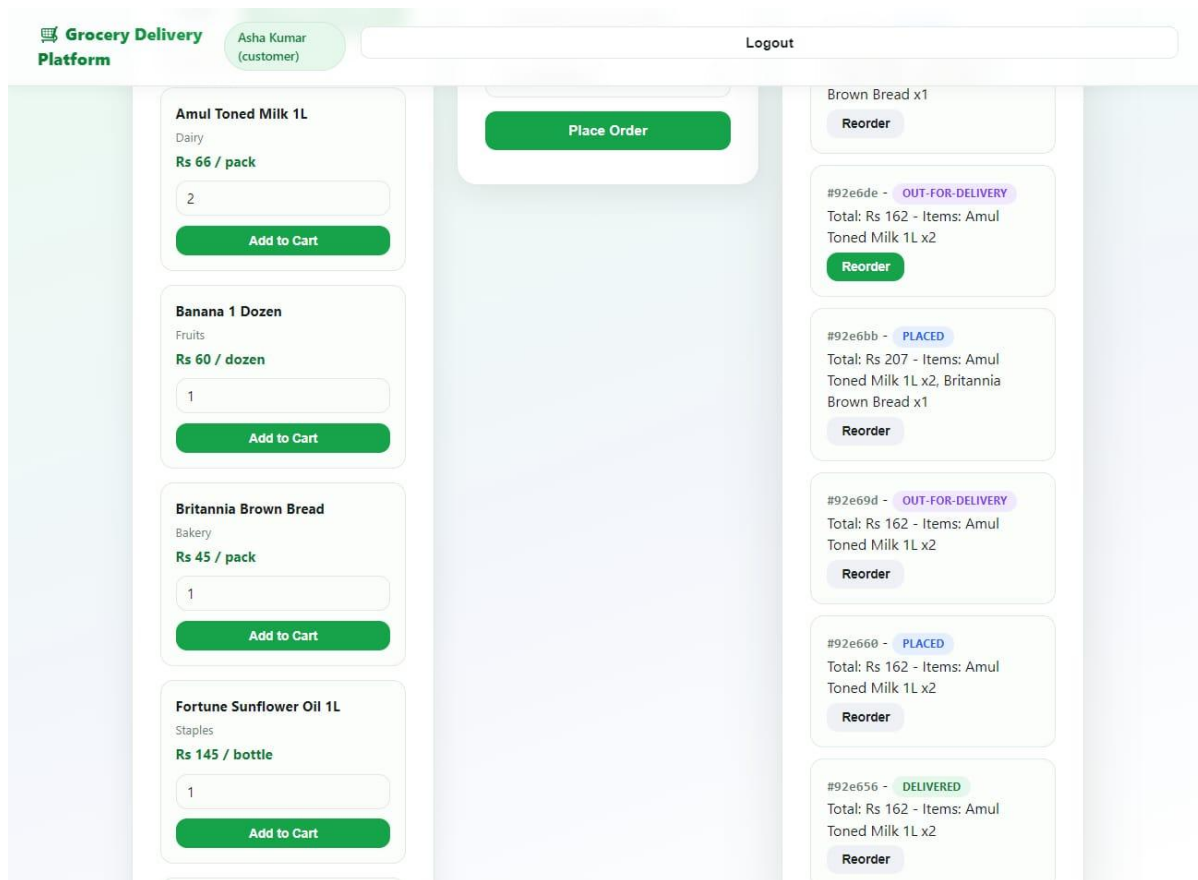
GET /api/orders/my	Customer	Order history
POST /api/orders/:id/reorder	Customer	Reorder a past order (re-validates stock & price)



Postman — GET /api/orders/my -> 200 Order history fetched successfully



Postman — POST /api/orders/:id/reorder -> 201 Order placed successfully (new_id)



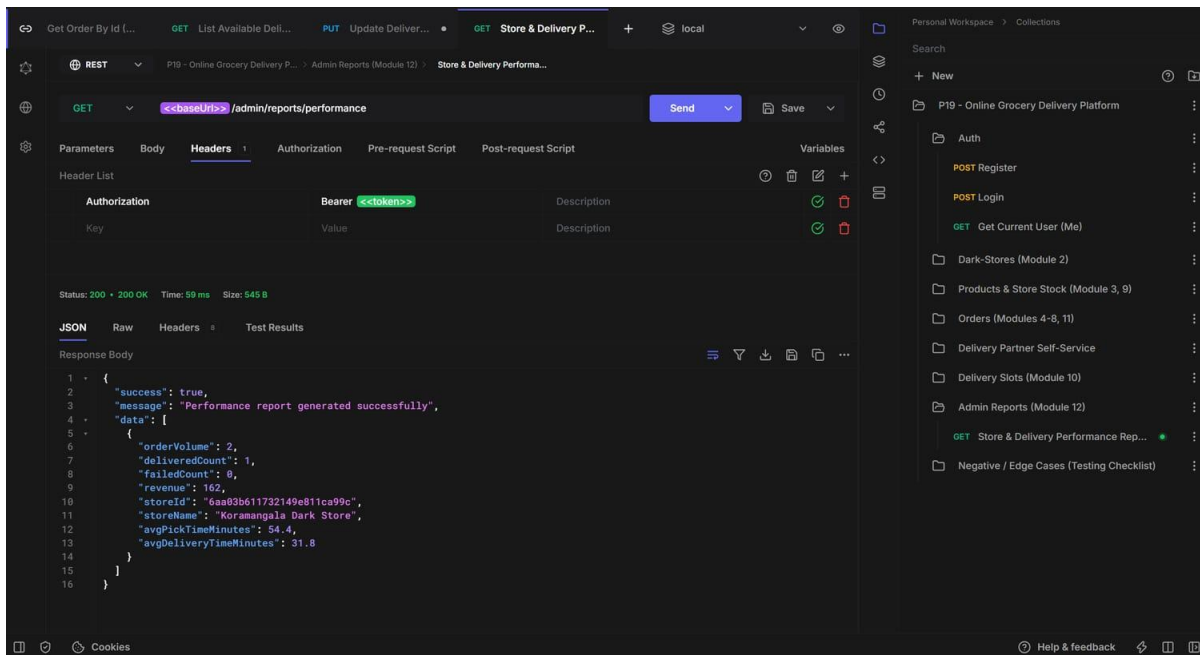
Demo UI — customer order history with per-order status and one-tap Reorder

4.12 Module 12 — Store & Delivery Performance Reports

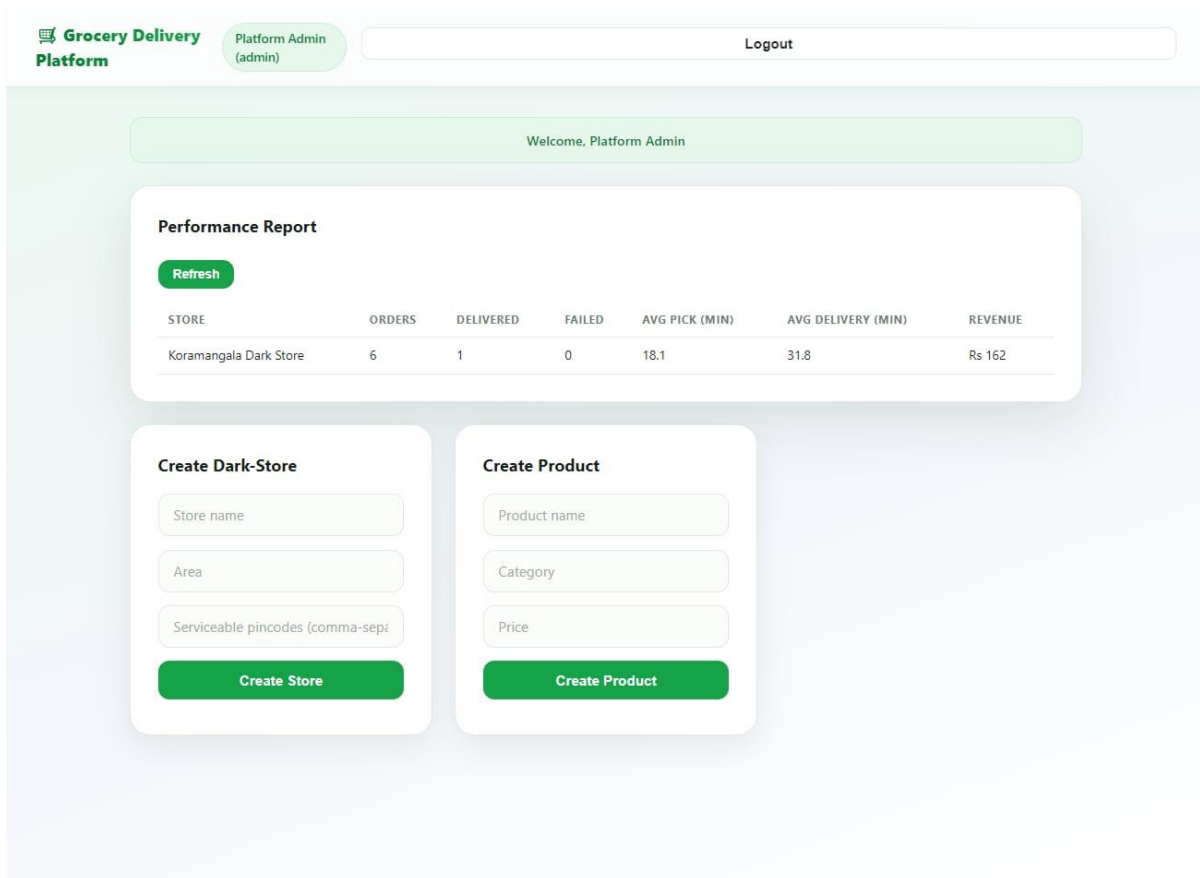
Admin-only. A single MongoDB aggregation pipeline computes, per store: order volume, delivered/failed counts, average pick time (createdAt -> packedAt), average delivery time (assigned event -> deliveredAt), and revenue (sum of totalAmount for delivered orders) — straight from timestamps already on each order, with no separate reporting table.

Endpoints:

GET /api/admin/reports/performance Admin Per-store volume, avg pick/delivery time, revenue



Postman — GET /api/admin/reports/performance -> 200 Performance report generated successfully



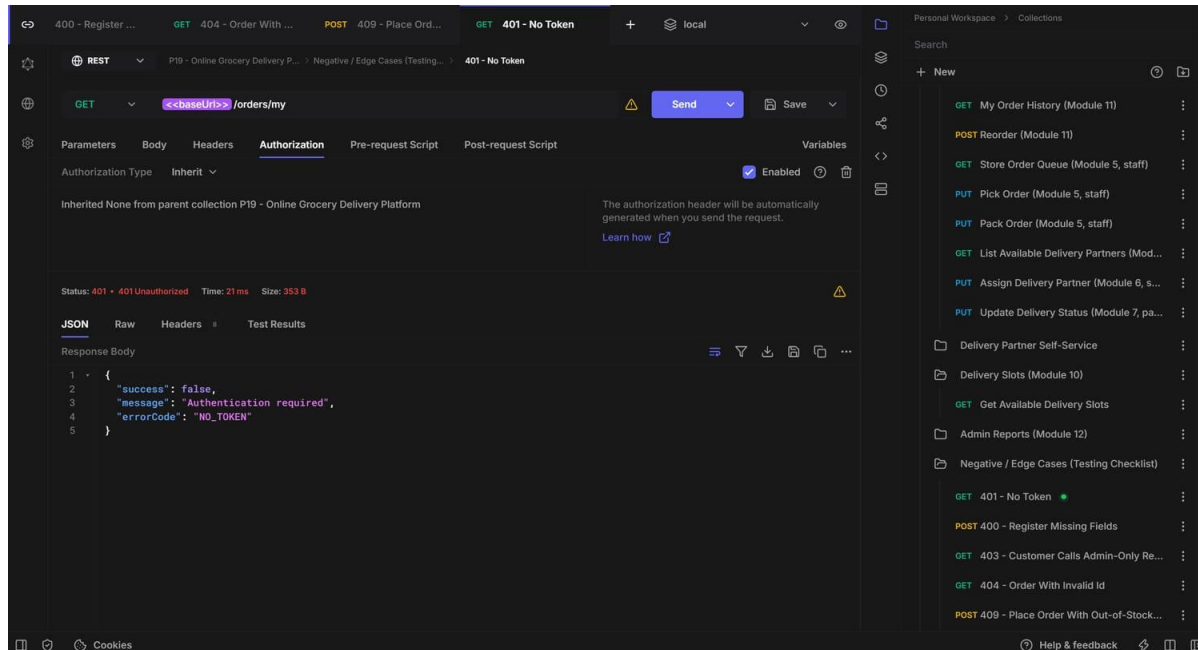
Demo UI — admin performance report (6 orders, 1 delivered, avg pick 18.1 min, avg delivery 31.8 min, revenue Rs 162)

4.13 Module 13 — Role-Based Access Control

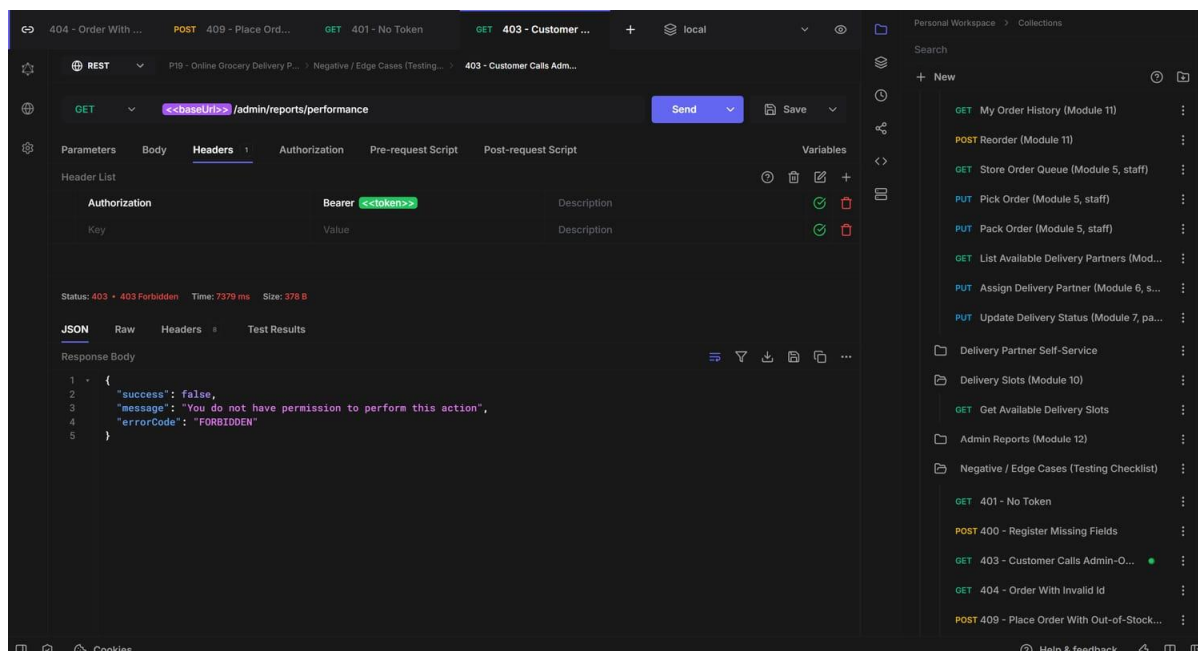
Enforced in three layers: (1) middleware/auth.js verifies the JWT and attaches the live user record (so a revoked/deactivated user is rejected even with a valid token); (2) middleware/authorize(...roles) is a per-route role allow-list; (3) record-level checks inside orderController.canAccessOrder restrict each role to the orders it is actually party to. Missing token -> 401; wrong role -> 403.

Endpoints:

```
protect -> 401 Authentication required (no/invalid token)
authorize('admin', ...) -> 403 You do not have permission to perform this action
canAccessOrder(order,user) -> 403 on another user's order even with a valid token
```



Postman — protected route with no token -> 401 Authentication required



Postman — customer token calling the admin report -> 403 Forbidden

5. Complete API Endpoint Reference

5.1 Auth

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register customer/staff/partner/admin
POST	/api/auth/login	Public	Login, returns JWT
GET	/api/auth/me	Any	Current authenticated user

5.2 Dark-Stores & Store Stock

Method	Endpoint	Auth	Description
POST	/api/stores	Admin	Create a dark-store
GET	/api/stores	Any	List dark-stores
GET	/api/stores/:id	Any	Get one store
PUT	/api/stores/:id	Admin	Update a store
DELETE	/api/stores/:id	Admin	Deactivate a store (soft delete)
GET	/api/stores/nearest?pincode=	Any	Nearest-store check by pincode
POST	/api/stores/:storeId/stock	Admin/Staff	Upsert stock for a product
GET	/api/stores/:storeId/stock	Admin/Staff	List a store's stock
PUT	/api/stores/:storeId/stock/:productId	Admin/Staff	Update quantity / reorderPoint
GET	/api/stores/:storeId/stock/alerts	Admin/Staff	Items at/below reorderPoint
GET	/api/stores/:storeId/orders?status=	Admin/Staff	Store picking / packing queue

5.3 Products

Method	Endpoint	Auth	Description
POST	/api/products	Admin	Create a product
GET	/api/products?category=&search=	Any	Browse / search products
GET	/api/products/:id	Any	Get one product
PUT	/api/products/:id	Admin	Update a product
DELETE	/api/products/:id	Admin	Deactivate a product

5.4 Orders

Method	Endpoint	Auth	Description
POST	/api/orders	Customer	Place an order (nearest-store + stock check)
GET	/api/orders/:id	Owner/Staff/Partner/Admin	Real-time order status + history

Method	Endpoint	Auth	Description
GET	/api/orders/my	Customer	Order history
POST	/api/orders/:id/reorder	Customer	Reorder a past order
PUT	/api/orders/:id/pick	Staff/Admin	Mark items being picked
PUT	/api/orders/:id/pack	Staff/Admin	Mark order packed
PUT	/api/orders/:id/assign	Staff/Admin	Assign a delivery partner
PUT	/api/orders/:id/status	Partner/Admin	Update delivery status

5.5 Delivery Partners, Slots & Admin Reports

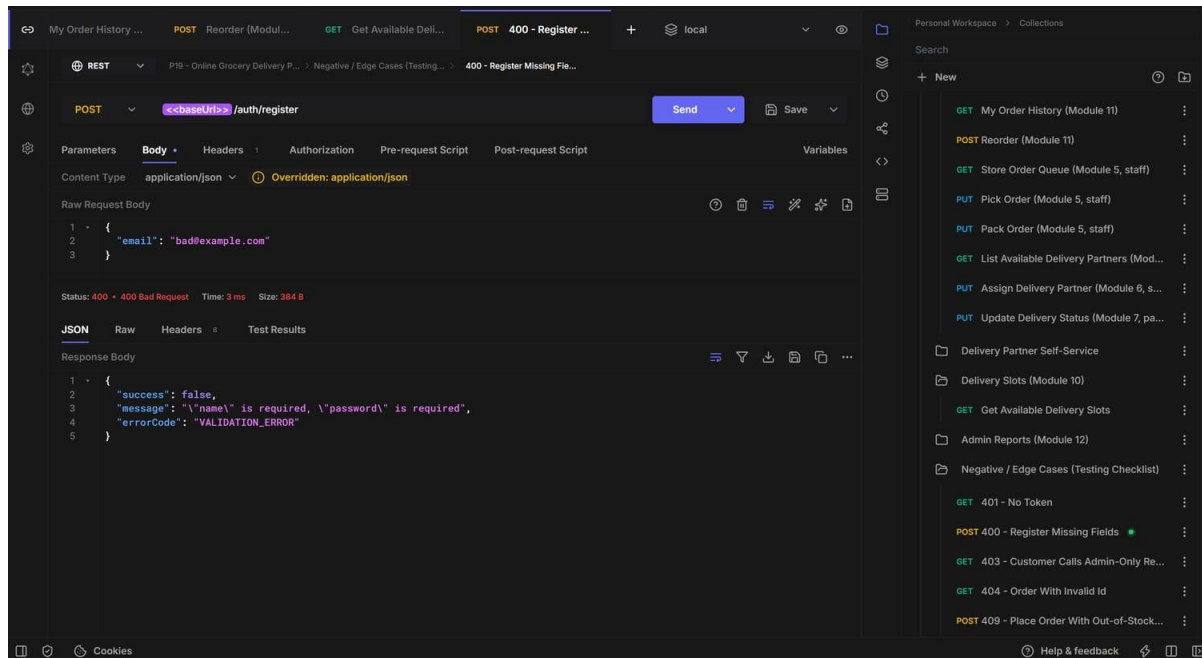
Method	Endpoint	Auth	Description
GET	/api/delivery-partners/available?storeId=	Staff/Admin	List available partners
GET	/api/delivery-partners/me	Partner	My partner profile
PUT	/api/delivery-partners/me/availability	Partner	Toggle availability
GET	/api/slots	Any	Bookable delivery windows (today/tomorrow)
GET	/api/admin/reports/performance	Admin	Per-store performance report

6. Testing, Validation & Error Handling

Every endpoint was exercised end-to-end with the Postman collection against the live backend, role by role (customer, staff, partner, staff, partner, customer, admin), reproducing the full order lifecycle. Beyond the happy path, the collection includes a dedicated set of negative / edge cases that assert the correct status code and error envelope.

6.1 Negative & Edge Cases

Status	Scenario	Response
400	Register with missing fields	"name" is required, "password" is required — errorCode VALIDATION_ERROR (Joi, abortEarly:false)
401	Protected route with no token	Authentication required — errorCode NO_TOKEN
403	Customer calls the admin-only report	You do not have permission to perform this action — errorCode FORBIDDEN
404	GET order with a non-existent ObjectId	Order not found — errorCode NOT_FOUND
409	Place order with quantity above stock	'<product>' is out of stock at <store> (requested N, available M) — errorCode OUT_OF_STOCK



400 — validation error, all missing fields reported at once





- `Joi schema per write route via validate() middleware: abortEarly:false reports every problem at`

- `asyncHandler` wraps every async controller so rejected promises reach the error handler.
- State-machine guard (`assertTransition`) rejects any illegal order status change with 409 `INVALID_TRANSITION`.

7. Setup & Run Instructions

1. Prerequisites: Node.js 18+ and npm; MongoDB (local install or a free MongoDB Atlas cluster).

2. Install dependencies:

```
cd grocery-delivery-platform/backend
npm install
```

3. Configure environment — copy `.env.example` to `.env` and set `MONGO_URI`, `PORT`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `SALT_ROUNDS`.

```
MONGO_URI=mongodb://127.0.0.1:27017/grocery_delivery
PORT=5000
JWT_SECRET=<long-random-string>
JWT_EXPIRES_IN=1d
SALT_ROUNDS=10
```

4. Seed demo data (2 stores, 6 products, per-store stock, one login per role — password `Passw0rd!` for all):

```
npm run seed
```

5. Run the server:

```
npm run dev      # nodemon (auto-restart)
# or
npm start
```

6. API is live at `http://localhost:5000/api` ; the demo frontend is served at `http://localhost:5000/` .

7. Import `postman/grocery-delivery-platform.postman_collection.json`, set `baseUrl` = `http://localhost:5000/api`, run `Auth > Login` — the token is captured automatically for every subsequent request.

7.1 Seed Accounts

Role	Email	Password
Admin	admin@grocery.test	Passw0rd!
Store Staff (Koramangala)	staff@grocery.test	Passw0rd!
Customer	customer@grocery.test	Passw0rd!
Delivery Partner	partner@grocery.test	Passw0rd!

8. Known Limitations

- Stock is decremented at order placement (not at pack time); a cancelled/failed order does not currently auto-restock.
- Stock decrements run as sequential `updateOne` calls rather than a MongoDB multi-document transaction, because a single-node local MongoDB has no replica set. On Atlas (always a replica set) this upgrades to a transaction with no other code change.
- Payment, SMS/email notifications, and maps/geolocation are out of scope and are not mocked beyond the nearest-store-by-pincode lookup.
- Delivery-slot capacity is not tracked — a window can be selected by unlimited customers; only the customer's own preferred window is recorded.

- The frontend is a minimal demo to make the live API demo easier to follow, not a polished consumer UI.

9. Learnings & Challenges

- Modelling the order lifecycle as an explicit state machine in one constant (`ORDER_STATUS_FLOW`) made the pick/pack/assign/deliver controllers tiny and impossible to misuse — every transition is validated against one table.
- Deciding embed vs. reference per field (not per collection) was the key database lesson: order line items must be an embedded frozen snapshot so past invoices never change, while stock must be its own referenced collection because it is updated on every single order.
- Two-layer RBAC (route-level role allow-list + record-level ownership check) is necessary — `authorize('storeStaff')` alone would still let one store's staff act on another store's orders without `canAccessOrder`.
- A single response envelope + one central error handler + `asyncHandler` removed almost all try/catch noise from controllers and guaranteed consistent error output for the Postman assertions.
- Running the whole collection role-by-role surfaced ordering bugs (e.g. a token overwritten by a later register call) that testing endpoints in isolation would have missed.
- Compound indexes were chosen to match the actual query shapes (customer history, store queue, partner deliveries) rather than added blindly per field.

10. Conclusion

P19 delivers a complete, working quick-commerce grocery backend: all 13 modules implemented and verified, a considered MongoDB schema with justified embed/reference choices and query-shaped indexes, a guarded order state machine, three-layer role-based access control, consistent validation and error handling, a full Postman collection, and a demo UI that exercises every role. The GitHub repository contains the full source, README, `.env.example` and Postman collection; `node_modules` and `.env` are git-ignored.

Repository: <https://github.com/Aaryan658/L-T-5th-sem-grocery-delivery-platform>